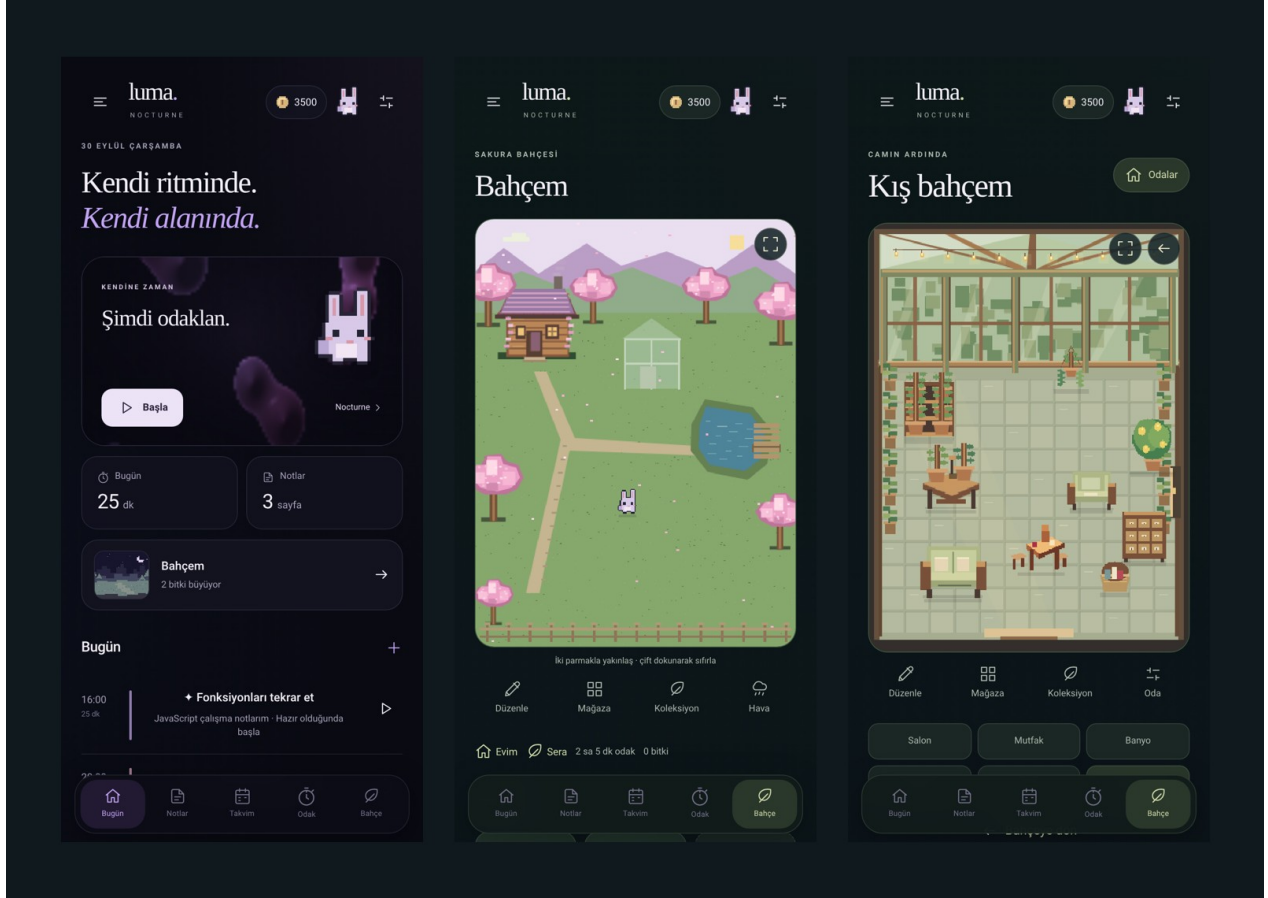


Sıfırdan koduna.

Görselli ürün tanıtımı, kullanım kılavuzu
ve geliştirici eğitimi



LumaNote 0.5.7 • Bugün, Sakura bahçesi ve kış bahçesi. Örnek kayıtlarla alınmış uygulama görünümüleri.

Bir uygulamayı anlamanın yolu

Önce ne yaptığını gör. Sonra hangi veriyi sakladığını öğren. En son o veriyi değiştiren kodu izle.

Kaynak sürüm: LUMA-0.5.7-HOME-GREENHOUSE

Belge tarihi: 1 Ekim 2026

Düzy: programlamaya ilk kez başlayan okuyucudan projeyi devralacak geliştiriciye kadar.

Bu rehberi nasıl okumalı?

Bu kitap, LumaNote'un bu sohbette paylaşılan son tam kaynak paketi üzerinden hazırlanmıştır: LumaNote_v057_Evim_ve_Kis_Bahcem_Tam_Proje.zip. Arşivdeki 0.5.7 kodları, yapılandırmalar, ses metaverileri ve test raporu incelenmiştir. Mac'inde sonradan yaptığın ve bu arşive girmemiş değişiklikler bu incelemenin kapsamında değildir.

Üç farklı anlatım türü

“Projede” dediğimiz açıklamalar dosyalardan ve çalışan tarayıcı önizlemesinden gelir. “Öğrenme örneği” başlıklı kodlar konuyu basitleştirmek için bu rehberde özel yazılmıştır; projenin eksiksiz işlevlerinin yerine geçmez. “Geliştirici notu” ise mevcut yapının sonuçlarını veya bir değişiklik yaparken düşünmen gereken noktaları anlatır.

Önce ürün turunu oku. Kod yazmayı hiç bilmiyorsan dil ve kavram bölümlerine geç. Sonraki mimari ve özellik bölümleri, gördüğün ekranın arkasında hangi dosyaların çalıştığını gösterir. Son bölümdeki küçük alıştırmalar gerçek notlarını, bahçeni veya jetonlarını değiştirmeden deneme yapmanı sağlar.

Görsellerin kaynağı

Ekranlar 0.5.7 paketindeki ONIZLEME.html kullanılarak Chromium'da alınmıştır. Notlar, süreler, isimler, bakiyeler ve oda yerleşimleri yalnızca öğretici örneklerdir. Şemalar, kodun işleyişini açıklamak için hazırlanmıştır; telefon ekranı değildir.

Bu belge neyi doğrulamıyor?

Bu çalışma bir mağaza onayı, güvenlik sertifikası veya fiziksel iPhone test raporu değildir. Kaynak doğrulayıcısı yeniden çalıştırıldı ve 62 kontrolü geçti; sayfa görselleri oluşturulurken yakalanmamış JavaScript hatası görülmedi. Paketle birlikte gelen 190 kontrollük rapor ayrı bir geçmiş test kayıdıdır. PDF motoru, gerçek bildirim teslimi, telefon klavyesi ve ses doğallığı yalnızca bir ekran görüntüsünden doğrulanamaz.

Kaynak izi [K01, K25, K26] package.json; README_TR.md; tools/verify.cjs; docs/TEST_RAPORU_057.md.

İçindekiler

01 Ürünü ve sayfaları tanı / 4

Bugün, notlar, PDF, takvim, odak, bahçe, sera, ses, yedek ve ayarlar.

02 Diller ve temel kavramlar / 25

HTML, CSS, JavaScript, TypeScript; olaylar, türler, React ve geliştirme araçları.

03 Mimari ve veri yolculuğu / 33

Telefon kabuğu, kaynak üretimi, klasör haritası, S/UI, köprü ve kayıt.

04 Ekranın arkasındaki kod / 40

Not seçimi, sayaç, jeton, bitki, piksel çizim, kamera, odalar, ses ve yedek.

05 Geliştirme masası / 54

Başlatma, küçük değişiklik döngüsü, hata okuma, testler ve güvenli güncelleme.

06 Uygulamalı öğrenme / 59

Bağımsız HTML atölyeleri, tür alıştırması ve benzer uygulamayı sıfırdan kurma.

07 Devir ve yayın öncesi / 65

Kaynakta bulunanlar, bulunmayanlar ve ayrıca doğrulanacak işler.

08 Sözlük ve kendini dene / 66

Temel sözcükler ve cevaplı anlama soruları.

09 Kaynak haritası ve ek okuma / 69

K01–K29 dosya eşleştirmeleri; resmî başvuru belgeleri.

10 Öğrenmeye devam / 73

Kendi hızında çalışma planı ve kısa devir özeti.

Bölüm adları tıklanabilir. Sayfa altındaki K kodları, Kaynak Haritası bölümüne karşılık gelir.

LumaNote ne yapıyor?

LumaNote, düşünce ve çalışma düzenini tek bir kişisel alanda toplar. Not alırsın, bir işi takvime yerleştirirsin, odak oturumu başlatırsın ve tamamlanan çalışmanın karşılığını piksel bahçende görürsün. Uygulamanın görsel dili karanlık Nocturne arayüzünü, hareketli arka planları ve düzenlenebilir küçük dünyaları birlikte kullanır.



Ekrandaki döngü, ayrı verilerin birbirine kimliklerle bağlanmasıyla oluşur.

Şema 1. Temel kullanım döngüsü. Oklar, ekranlar arasındaki bağlantıyı gösterir.

Oyun kısmı ile çalışma kısmı nasıl birleşir?

Bahçe ayrı bir oyun sunucusuna bağlı değildir. Kaydedilen odak süresi uygulama durumuna işlenir. Aynı süre jeton hesabında ve bitkilerin gelişiminde kullanılır. Bir hayvan veya dekor satın almak bir alışveriş kaydı oluşturur. Onu sahneye yerleştirmek ise ayrıca konum kaydı oluşturur. Bu yüzden satın almak, yerleştirmek ve çalışmak üç farklı işlemdir.

Ürünün sınırı

Bu pakette LumaNote kullanıcı hesabı, uzak veritabanı, çok cihazlı otomatik eşitleme veya gerçek para ile alışveriş sistemi bulunmuyor. Expo hesabı geliştirme aracına aittir; LumaNote içinde bir kullanıcı profili sunucusu olduğu anlamına gelmez. Notlar ve ilerleme esas olarak cihazın yerel kayıt alanında tutulur. Çalışma kaydı da kişinin gerçekten ders çalıştığını ölçen veya doğrulayan bir sistem değildir.

Tek cümleyle

LumaNote, yerel kayıt tutan bir not ve odak uygulamasını, çalışma süresiyle gelişen kişisel bir piksel dünya ile birleştirir.

Kaynak izi [K03, K05, K10, K11] core.js; NativeServices.ts; stable-focus.js; world.js.

İlk kullanım: bir düşünceden bahçeye

İlk kez kullanan biri için bütün ayarları aynı anda öğrenmek gerekmez. Aşağıdaki örnek yol, uygulamanın ana parçalarının neden birlikte bulunduğunu gösterir. Bu bir öğrenme senaryosudur; ekrandaki örnek notların senin kişisel verin olduğu anlamına gelmez.

Adım	Uygulamada yaptığın	Arka planda oluşan
1. Not al	Notlar → + ile “Bugünkü konum” başlıklı not oluştur.	S.notes dizisine kimliği olan bir not eklenir.
2. İstersen planla	Notu bir çalışma planına bağla veya Takvim’den zaman seç.	Plan, notun tamamını kopyalamak yerine noteld saklar.
3. Tohum ek	Bahçe → Yetiştir’den boş saksıya tohum seç.	Ekim anındaki toplam çalışma saniyesi kaydedilir.
4. Çalış	Odak bölümünde bir yöntem seç ve Başla’ya dokun.	Başlangıç, bitiş ve kalan süreyle etkin sayaç kurulur.
5. Bitir ve kaydet	Süre dolduğunda veya kısmi çalışmayı kaydederek oturumu bitir.	Tek bir geçmiş kaydı ve buna karşılık gelen çalışma kredisi işlenir.
6. Dünyanı geliştir	Jetonla parça satın al, yerleştir; bitki hazırsa bahçeye taşı.	Alışveriş, sahiplik ve konum bilgileri ayrı alanlarda korunur.

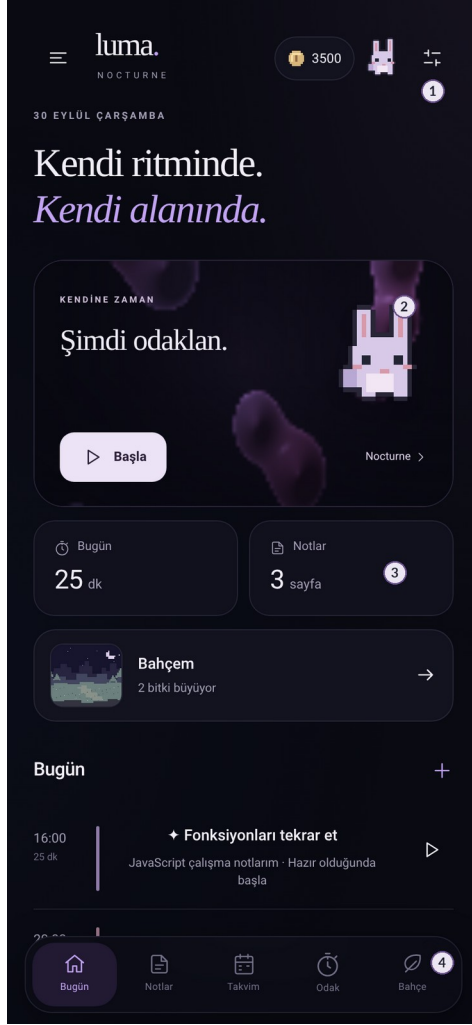
Küçük ama önemli farklar

Duraklatmak oturumu tamamlamak değildir. Bir notu arşivlemek çöpe taşımak değildir. Özel bir hava modu satın almak bütün dekorları otomatik vermek değildir. Bu ayrımları öğrenmek, hem uygulamayı kullanırken hem kod yazarken yanlış varsayımları engeller.

Her ekrandaki önemli düğmeyi üç soruyla incele: Hangi veri değişiyor? Değişiklik ne zaman kaydediliyor? İşlem başarısız olursa kullanıcı ne görüyor? Bu sorular kitabın sonraki bölümlerindeki bütün kod incelemelerinde tekrar kullanılacaktır.

Kaynak izi [K07, K10, K11, K19] createNote; startFocus/endFocus; plantSeed/harvestPlant; planSheet.

Bugün: uygulamanın başlangıç noktası



Ekranın amacı

Bugün sayfası, uygulamanın bütün özelliklerini aynı anda göstermeye çalışmaz. Kısa bir odak başlangıcı, günün özeti ve planlarıyla diğer bölümlere geçiş sağlar.

1 • Üst araçlar

Jeton simgesi mağazaya, piksel hayvan yol arkadaşına, ayar çizgileri ayarlara açılır. Aynı başlık dili başka sayfalarda da kullanılır.

2 • Başlama kartı

Büyük kartın temel eylemi çalışmayı başlatmaktır. Tema ve seçili hayvan bu kartın görünümüne eşlik eder. Kartın kendisi ayrı bir veri kaydı değildir.

3 • Özet ve planlar

Bugünkü çalışma süresi geçmiş oturumlardan hesaplanır. Not sayısı ve günün takvim öğeleri mevcut verilerden gelir; ekranda görülen rakamları tek tek elle saklamak gerekmez.

4 • Alt gezinme

Bugün, Notlar, Takvim, Odak ve Bahçe beş ana giriş noktasıdır. Kodda bunlar home, notes, calendar, focus ve garden anahtarlarıyla temsil edilir.

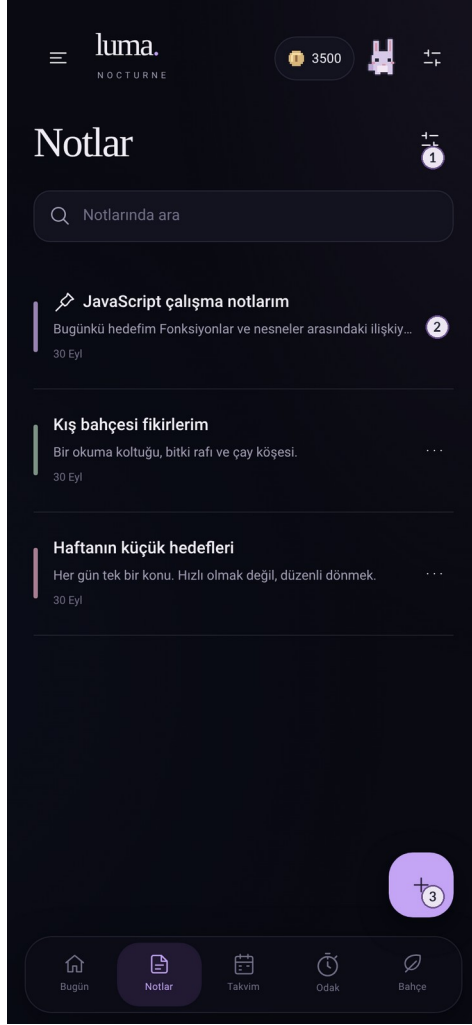
Kodda karşılığı

Temel görünüm screens.js içindeki renderHome ile başlar. Sonraki katmanlar kartları ve hayvan gösterimini geliştirebilir. Gezinme, core.js içindeki route/render akışı üzerinden yürür.

Şekil 1. Bugün sayfası. Numaralar rehber açıklamalarıdır; uygulamanın özgün arayüzüne dahil değildir.

Kaynak izi [K03, K07] core.js → header, renderNav, route; screens.js → renderHome.

Notlar: hızlı yakalama ve toplu düzen



1 • Arama ve filtre

Not başlığı ve metnindeki içeriği aramak için üst arama alanını kullanırsın. Filtre menüsü görünümü daraltır. Ekranda bir notun bulunmaması mutlaka silindiği anlamına gelmez.

2 • Not satırı

Başlık, kısa metin özeti ve güncelleme bilgisi bir notun listede tanınmasını sağlar. Sabitleme, arşiv ve çöp kutusu farklı durum alanlarıdır.

3 • Sabit yeni not düğmesi

Sağ alttaki +, listedeki not sayısına bağlı olarak aşağı kaçmaz. Yeni boş not açılır; şablon seçmek zorunlu ilk adım değildir.

Basılı tutarak seçim

Bir nota uzun basınca seçim modu başlar. Diğerlerine dokunarak seçimini büyütebilirsin. Tümünü seç, arama ve filtreyle görünür olan sonuçlara uygulanır.

Silme ne yapar?

Normal işlem notu çöp kutusuna taşır. Kalıcı silme ayrı onay ister. Bir onay menüsü açıldığında arkadaki toplu işlem çubuğu gizlenir; seçimin Vazgeç ile korunması amaçlanır.

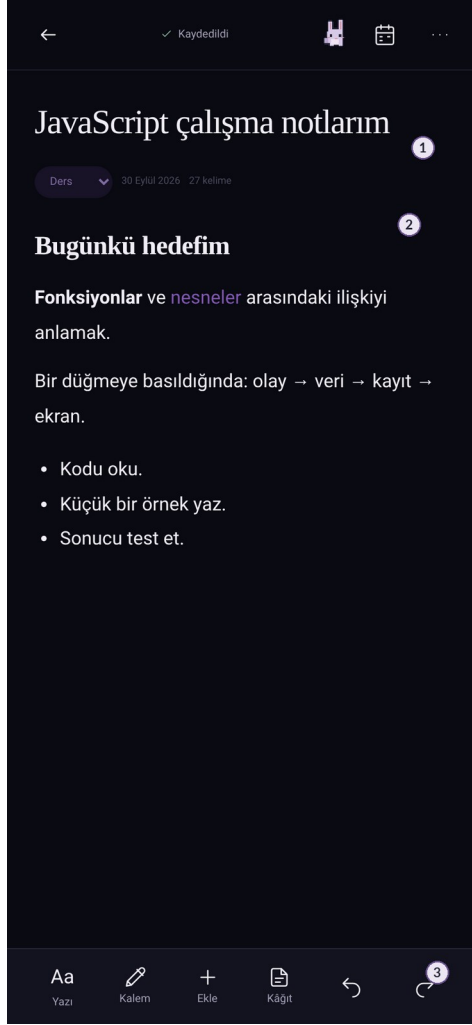
Kodda karşılığı

living-notes.js sade listeyi kurar. polish-notes.js seçim kümesini ve toplu işlemleri yönetir. haven-shell.js, onay menüsü sırasında arka katmanı etkileşime kapatır.

Şekil 2. Örnek not listesi. Bu ekrandaki başlıklar öğretim için hazırlanmıştır.

Kaynak izi [K07, K08] living-notes.js; polish-notes.js → beginNoteSelection054, bulkApply054; haven-shell.js.

Not düzenleyici: içerik ve araçlar



1 • Başlık ve kategori

Notun başlığı, gövdesi ve kategorisi ayrı alanlardır. Listeye döndüğünde aynı notun kısa görünümü hazırlanır; ekran görüntüsü notun kendisi olarak saklanmaz.

2 • Zengin metin

Metni seçerek kalın, italik, altı çizili veya renkli hale getirebilirsin. Biçimlendirme seçili aralığa uygulanır. Bu nedenle metin seçiminin araç düğmesine dokunurken korunması önemlidir.

3 • Araç çubuğu

Yazı biçimi, Kalem, Ekle, Kâğıt ve geri al/yinele araçları aynı editörde bulunur. Ekle bölümünden dosya ve tablo gibi içerikler eklenebilir.

Kaydedildi göstergesi

Yazdığın her harfte dosya işlemi başlatmak yerine değişiklikler kısa bir bekleme sonrasında kaydedilir. Gösterge, kaydetme durumunu görünür kılar. Editörden çıkışta içerik ayrıca toplanır.

Bir sınır

Bu ekran Apple Notlar'ın kodunu kullanmaz. HTML düzenleme alanı ve projeye ait JavaScript işlevleriyle oluşturulmuştur. Klavye ve seçim tutamaçları telefonun WebView davranışına da bağlıdır.

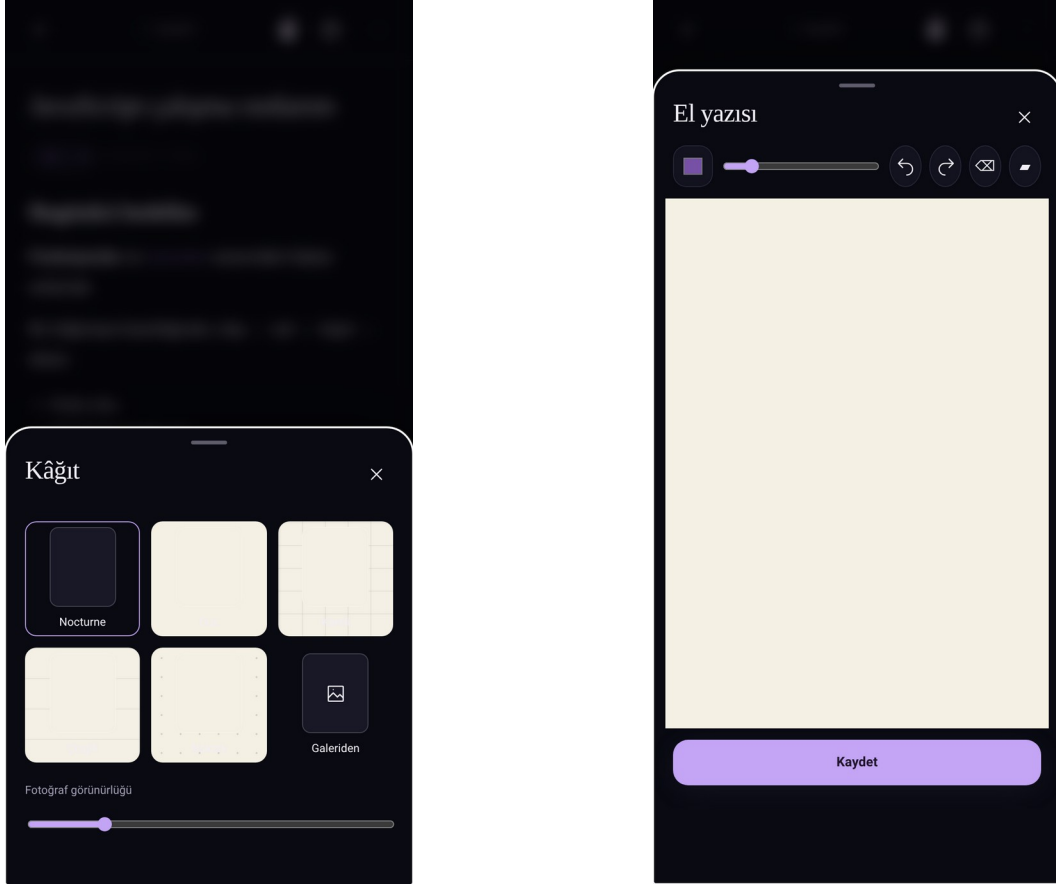
Öğrenme görevi

Bir kelime seç; kalınlaştır ve rengini değiştir. Notu kapatıp yeniden aç. Görsel değişikliği değil, kayıt sonrası korunmasını gözlemler.

Şekil 3. Düzenleyici. Görünümün büyük bölümü HTML ve CSS; davranış JavaScript ile sağlanır.

Kaynak izi [K07, K08] editor.js; editor-v04.js; polish-notes.js → captureRange, format, writingPanel054.

Kâğıt, el yazısı ve tablolar



Şekil 4. Kâğıt seçimi ve ayrı el yazısı çalışma alanı.

Kâğıt seçimi notun arka plan görünümünü değiştirir: koyu, düz, kareli, çizgili, noktalı veya bir resim. Resim seçeneği, bir ekin kimliği ve saydamlık ayarıyla ilişkilidir. Arka planı değiştirmek metni veya çizimleri silme işlemi değildir.

El yazısı, metnin üzerine rastgele piksel boyamak yerine ayrı bir çizim alanında tutulur. Çizgi; renk, kalınlık, araç ve nokta dizisinden oluşur. Kalemle yaptığın bir hareketin noktaları kayıt haline gelir. Silgi, burada gerçek bir fiziksel kalem değil, bu nokta kayıtları üzerinde çalışan bir işlemdir.

Tablo ekleme ise farklıdır: satır ve sütunlardan oluşan HTML tablo yapısı metin düzenleyicisine eklenir. Hücreye yazı yazılabilir. Bir tablonun ekran görüntüsünü yapıştırmakla, düzenlenebilir tablo oluşturmak aynı şey değildir.

Yanlış beklentiyi önle

Bu sürüm el yazısını otomatik metne dönüştüren bir tanıma modeli içermez. Çizim kaydı ile klavyeden girilen metin ayrı veri türleridir.

Kaynak izi [K09] living-notes.js → paperSheet, tableOptions, openInk, drawStrokes; living-model.js → normalizeInk.

PDF: orijinal belge ve işaret katmanı

Bir PDF eklediğinde önce ek dosyanın kendisi saklanır. Açıldığında PDF sayfası bir çizim yüzeyine dönüştürülür. Kalem, vurgulama ve yazılı not kutuları bunun üstündeki ikinci katmanda tutulur. Böylece orijinal belgeyi değiştirmeden çalışma yapılabilir.



Şema 2. PDF çalışma alanında orijinal, düzenlenebilir işaret ve paylaşılan kopya ayrımı.

Kullanım mantığı

Not düzenleyicisinde PDF ekle, ekin üzerine dokun, sayfayı seç ve uygun aracı kullan. Gezin aracı belge üzerinde hareket etmek içindir; Kalem çizmek içindir. İkisini ayırmak, yakınlılaştırırken yanlışlıkla çizgi oluşmasını önler. Metin notu eklemek için metni yazıp sayfada konum seçilir.

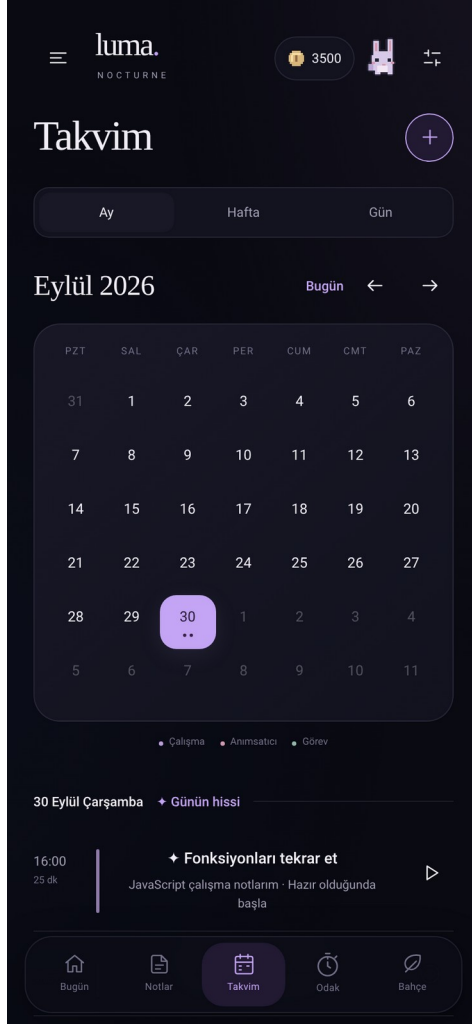
İşaretleli kopyayı paylaşırken yeni PDF, sayfa görüntüleri üzerinden üretilir. Bu nedenle orijinalin seçilebilir/aranabilir metni ve bağlantıları dışa aktarılan görüntü tabanlı kopyada korunmaz. Orijinal dosya yerinde kalır. Yeniden düzenlenebilen çizimler uygulamanın veri kaydında ayrıca bulunur.

Kaynakta görülen sınırlar

PDF seçimi en fazla 25 MB, belge görüntüleme en fazla 200 sayfa olarak sınırlandırılmıştır. Şifre korumalı PDF için şifresiz çalışma kopyası istenir. PDF kitaplıkları kurulumda hazırlanır; 0.5.7 raporu fiziksel cihazda PDF açma ve dışa aktarma doğrulaması yapıldığını söylemez.

Kaynak izi [K05, K09, K25] NativeServices.ts → copyPicked; living-pdf.js → ensurePdfLibraries, openPdf, exportAnnotatedPdf; test raporu.

Takvim: bir işi zamana yerleştirmek



Görünüm

Ay, hafta ve gün düzenleri aynı plan verisinin farklı gösterimleridir. Bir takvim kutusu veri değildir; belirli tarihe düşen planların görünümüdür.

Plan türleri

Çalışma, anımsatıcı, görev ve etkinlik seçenekleri vardır. Çalışma planı süre ve isteğe bağlı not bağlantısı taşır. Görev tamamlanması, odak süresi kazanımıyla aynı işlem değildir.

Tekrar

Günlük, hafta içi veya haftalık planlar için asıl başlangıç günü korunur. Tekrarlayan planın belirli bir gündeki tamamlanması, bütün tekrarların tamamlanması anlamına gelmez.

Notla bağlantı

Bir plan notla saklayarak bir nota bağlanabilir. Bu kimlik, notun başlığı değişse bile bağlantının korunmasına yarar. Programlamada iki kayıt arasındaki ilişkiyi isim yerine kimlikle kurmak önemlidir.

Bildirim

İzin verilirse uygulama yerel anımsatıcılar planlar. Takvimde planın görünmesi ile telefonun o saatte bildirim teslim etmesi farklı şeylerdir; ikincisi işletim sistemi koşullarına bağlıdır.

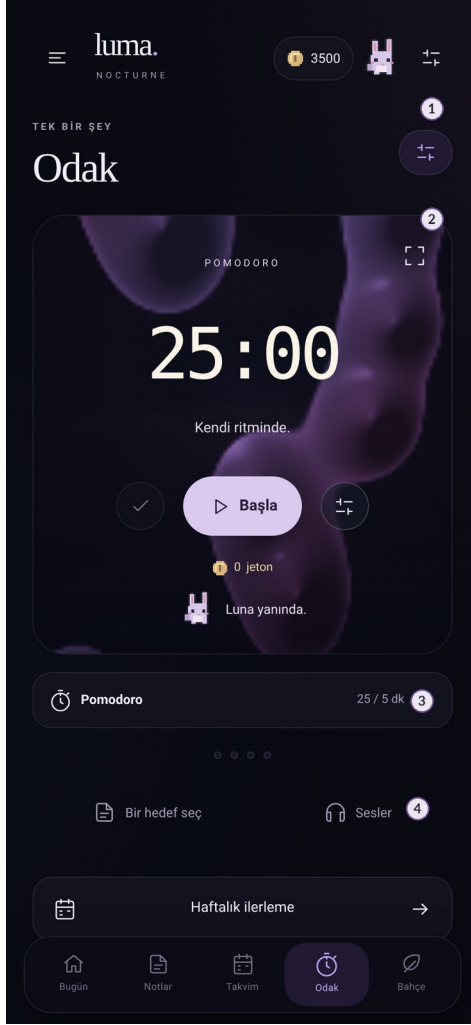
Öğrenme görevi

Bir çalışma planı oluştur, bir nota bağla ve tek bir gün için tamamla. Sonra aynı planı haftalık tekrar olarak incele. `completed` ile `completedDates` alanlarının neden ayrıldığını düşün.

Şekil 5. Takvim görünümü. Tarihler ve planlar yalnızca örnek veridir.

Kaynak izi [K19, K05] `screens.js` → `renderCalendar`; `planner.js`; `NativeServices.ts` → `occurrences`, `reconcile`.

Odak: zamanı görünür kılmak



1 • Oturum ayarları

Süre, yöntem, mola ve tur ayarları başlangıçtan önce düzenlenebilir. Hedef seçmek oturuma bir başlık veya not bağlantısı verebilir.

2 • Sayaç ve tema

Sayaç, tema çiziminin üzerinde gösterilir. Görsel animasyon ayrı, süre hesabı ayrıdır. Animasyonun daha yavaş çalışması odak süresini daha yavaş saydırmamalıdır.

Başlat, duraklat, kaydet

Başla yeni oturum oluşturur. Duraklat kalan zamanı dondurur. Bitir/kaydet gerçek geçen süreyi oturum geçmişine aktarır. Duraklatmanın kendisi jeton kazandırmaz.

Yol arkadaşın

Seçili hayvan ve verdiği isim normal odak görünümünde sana eşlik eder. Tam ekranda arayüz sadeleştirilir; bu görsel süsler sürekli gösterilmez.

3 • Yöntem kartı

Pomodoro veya diğer hazır çalışma düzenlerini buradan seçersin. Hazır değerler başlangıç düzenidir; uygulamanın amacı zorunlu tek bir ritim dayatmak değildir.

4 • Ses ve değerlendirme

Ses paneli ayrı kanalları karıştırır. Haftalık ilerleme de geçmiş kayıtlarını özetler. İkisi sayaçtan farklı modüllerdir, aynı sayfadan erişilebilirler.

Şekil 6. Normal odak ekranı; 25 dakikalık örnek başlangıç.

Kaynak izi [K10, K18, K22] living-focus.js; stable-focus.js; Scene; AudioEngine.

Hazır çalışma düzenleri

Aşağıdaki değerler araştırma sonucu “herkes için en iyi süreler” olarak sunulmuyor. Bunlar 0.5.7 kodundaki STUDY_METHODS tablosunun başlangıç ayarlarıdır. Yöntemin adını görmek ile o yöntemin bütün kuramsal ayrıntılarının yazılımla uygulanması aynı şey değildir.

Kart	Çalışma / kısa mola	Uzun mola / tur	Kod anahtarı
Pomodoro	25 / 5 dakika	20 dakika / 4 tur	pomodoro
52 / 17	52 / 17 dakika	17 dakika / 1 tur	rule5217
Zaman kutusu	45 / 10 dakika	10 dakika / 1 tur	timebox
Flowtime	İleri sayım / önerilen mola	10 dakika başlangıç / 1 tur	flow
Önce zor iş	45 / 10 dakika	10 dakika / 1 tur	frog
90 dakika	90 / 20 dakika	20 dakika / 1 tur	deep90
Kendi ritmim	50 / 10 dakika	20 dakika / 4 tur	custom

Mola geçişi

Oturum kaydı tamamlandıktan sonra bir sonraki aşama S.practice.pending içinde tutulur. Bu, “şimdi mola öneriliyor” ile “mola sayacı çalışıyor” durumlarını ayırır. Otomatik mola veya otomatik çalışma seçenekleri kapalı olduğunda yeni aşama kullanıcının eylemini bekler.

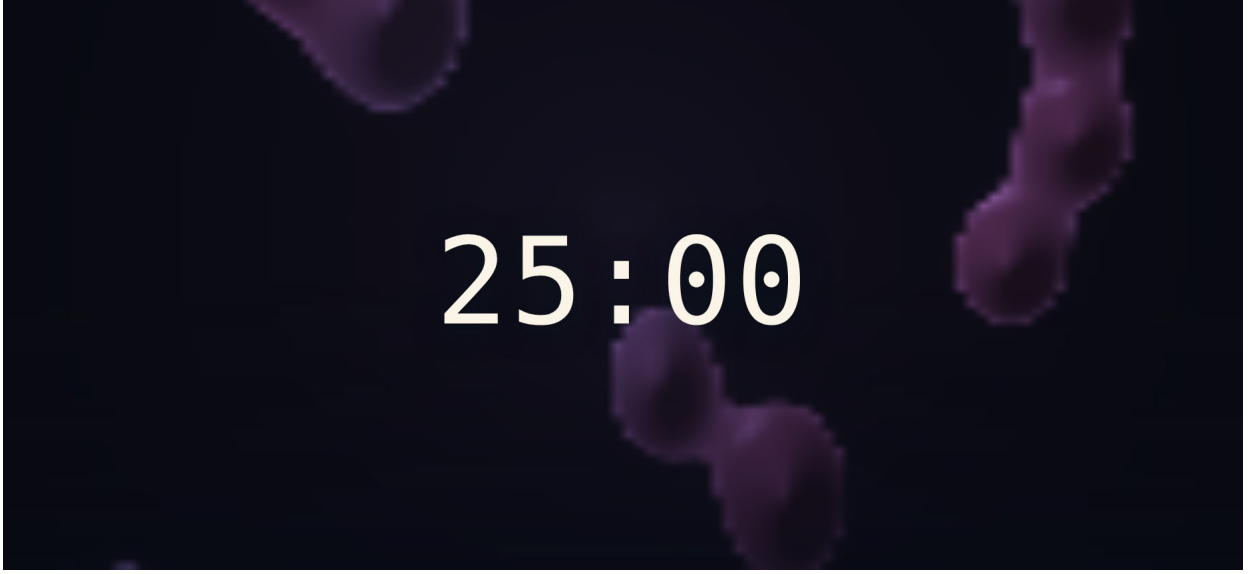
Flowtime görünümde ileri sayar; kaynakta kontrolsüz sonsuz sayaç değil, 720 dakikalık üst sınırlı bir oturum üzerinden hesaplanır. Mola önerisi nextBreakMinutes işlevinden gelir ve ayarlarla değişebilir. Sabit süreli yöntemler ise bitiş zamanına yaklaşan geri sayımı gösterir.

Geliştirici bakışı

Bu kartlar, arayüzde birbirine benzeyen ama veri tarafında farklı alanlar taşıyan bir “yapılandırma tablosu” örneğidir. Yeni kart eklemek sadece adını yazmak değildir: başlangıç değerleri, açıklama, ayarlar, kayıttan geri okuma ve test de düşünülmelidir.

Kaynak izi [K10, K11] living-model.js → STUDY_METHODS; living-focus.js → nextBreakMinutes; stable-focus.js → startFocus.

Tam ekran ve yatay odak



Şekil 7. Aynı uygulama HTML'inin yatay tam ekran düzeni. Bu görüntü fiziksel telefonun yön sensörü testi değildir.

Tam ekran görünümünün amacı daha fazla bilgi eklemek değil, dikkat dağıtan parçaları azaltmaktır. Normal durumda sayaç ve seçilen arka plan kalır. Ekran dokunulduğunda kontrol düğmeleri kısa süreliğine görünür. Bu, kontrolleri tamamen erişilemez yapmadan sade bir çalışma yüzeyi oluşturur.

Üç farklı sayaç görünümü

Kodda simple, pixel ve flip seçenekleri bulunur. Bunlar aynı zaman bilgisinin farklı görsel sunumlarıdır. Görünümü değiştirmek yeni bir odak oturumu açmamalı veya kalan süreyi sıfırlamamalıdır. Bu ayrım “veri ile sunumu ayırmak” ilkesinin somut örneğidir.

Dikey / yatay geçiş

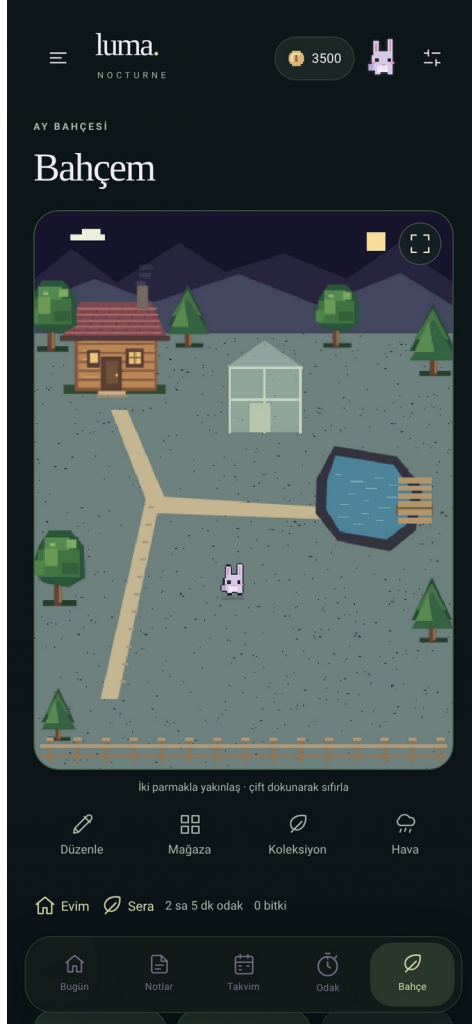
Web arayüzü yerleşimi CSS ile düzenler. Telefonun yön kilidi ise App.tsx üzerinden expo-screen-orientation servisine iletilir. Durum çubuğunun gizlenmesi de native taraftadır. İyi görünen bir tarayıcı ekranı bu native işlemlerin gerçek cihazda eksiksiz çalıştığını kanıtlamaz.

Bahçeyi arka plan olarak kullanmak

Bahçem teması seçilirse aynı kişisel dünyanın pasif bir görünümü kullanılır. Odak sırasında sahne düzenleme hareketleri kapalıdır. Böylece çalışma sırasında parmağınla yanlışlıkla bir mobilyayı taşımazsın.

Kaynak izi [K02, K10, K12] App.tsx → displayMode; living-focus.js → focusScene, showCinemaControls; pasif bahçe.

Bahçem: canlı sahne ve düzenleme



Dikey dünya

Bahçe sayfasındaki büyük sahne bir video veya sabit resim değildir. Canvas üzerine çizilen zemin, yapılar, hayvanlar, eşyalar ve hava parçacıklarından oluşur.

Yakınlaştır ve gezin

İki parmakla yakınlaştırma, yakinken tek parmakla gezinme sağlar. Kamera sınırları görüntünün dışına boş alan taşmamasını hedefler. Boş zemine çift dokunmak ölçeği başlangıca getirir.

Düzenle

Düzenleme modunda bir öğe seçilir; yön okları veya sürüklemeye taşınır. Kamera hareketi ile eşya hareketi farklı işlemlerdir. Seçim durumunun açıkça tutulmasının nedeni budur.

Yapılara giriş

Kulübe salonla başlayan ev alanına; sera ise kış bahçesine açılır. Ekran değişse de bütün sahiplik ve yerleşim bilgileri aynı yerel uygulama durumunda durur.

Sahnenin altı

Mağaza, Koleksiyon, Hava, Evim ve Sera girişleri bulunur. Yetiştir ve Anı bahçem bölümleri sayfanın devamındadır. Sahneye dokunmakla sayfayı aşağı kaydırmak karıştırılmamalıdır.

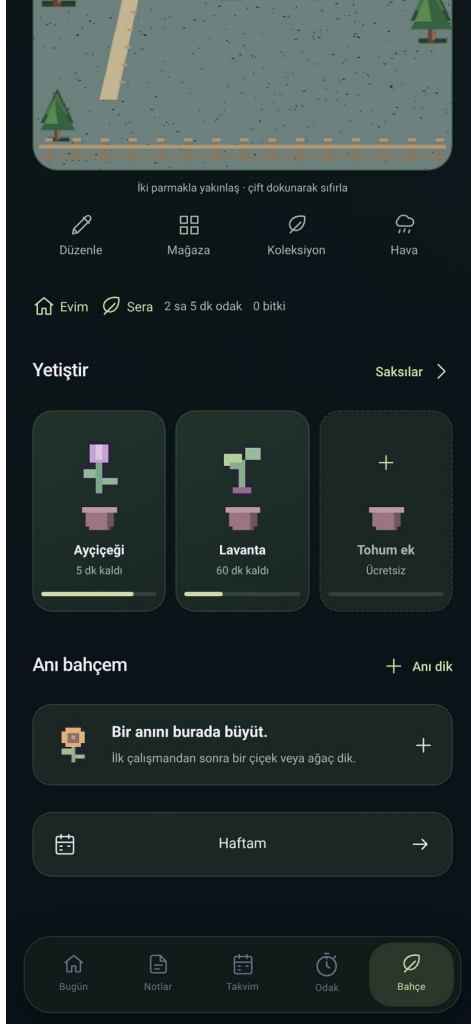
Öğrenme görevi

Bir eşyayı satın al. Yerleştirmeden önce koleksiyonda olduğunu, sahnede olmadığını kontrol et. Sonra yerleştir ve taşı. "Sahiplik" ile "konum" arasındaki farkı gözlemler.

Şekil 8. Büyük, dikey bahçe alanı; örnek Ay Bahçesi görünümü.

Kaynak izi [K12, K17] LivingGarden sınıf zinciri; haven-shell.js → renderGarden; world.layout ve living.placement.

Yetiştir ve Anı bahçem



Üç saksı

Saksılar bahçe sahnesinin altında ayrı büyüme kartlarıdır. Boş saksıya ücretsiz tohum seçilir. Kart, bitkinin mevcut aşamasını ve kalan odak süresini gösterir.

Gerçek ilerleme kaynağı

Bitki duvar saatinde beklediğin için değil, ekimden sonraki kaydedilmiş çalışma sürenle büyür. Mola süresi gelişime dahil olmaz. Büyüme için jeton bakiyesini kontrol etmek yanlış olur.

Sulama

Sulama bir görsel geri bildirimdir. Çalışmayla bir aşama değiştiğinde su damlaları ve gelişim hareketi sıraya alınır. Tekrar tekrar sulama, sonsuz süre veya jeton üretmez.

Bahçeye dik

Bitki tamamlandığında hasat edilip yerleştirilebilir bir bitkiye dönüşür. Saksı boşalır. Yeni bir tohum aynı saksıda yetiştirilebilir; eski bitki bahçede kalır.

Anı bitkisi

Anı bahçem ayrı bir anlam katmanıdır: isim verilmiş bir çiçek veya ağaç bir olayı temsil eder. Yeri düzenlenebilir; saklandığında anı kaydının kendisi korunabilir.

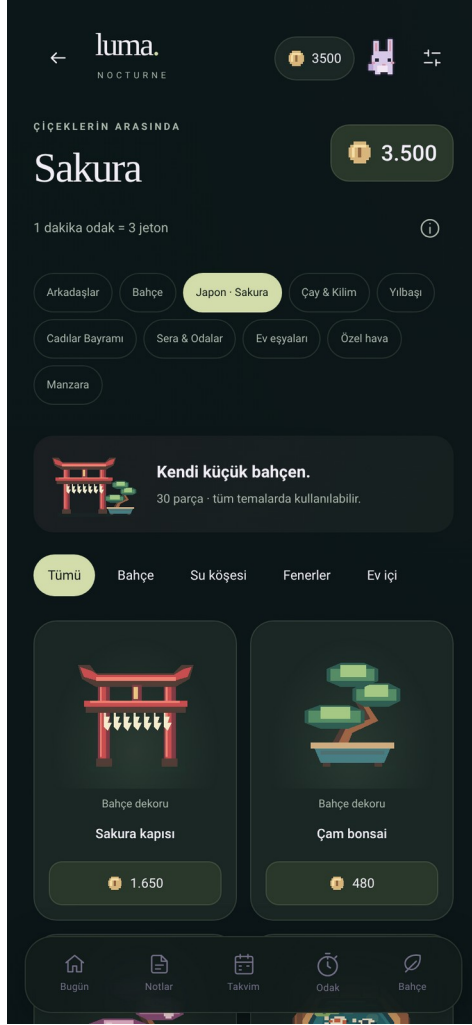
Kodda karşılığı

world.plots etkin saksıları, world.harvested tamamlanmış bitkileri, living.memorials anıları tutar. Bunlar benzer görünebilir ama farklı yaşam döngülerine sahip kayıtlardır.

Şekil 9. Bahçe sayfası aşağı kaydırıldığında Yetiştir ve Anı bahçem görünür.

Kaynak izi [K11, K12, K16] plantProgress; queueGrowth; harvestPlant; memorySection056; memorialForm056.

Mağaza: katalog, sahiplik ve yerleştirme



Kategori sistemi

Arkadaşlar, Bahçe, Japon · Sakura, Çay & Kilim, Yılbaşı, Cadılar Bayramı, Sera & Odalar, Ev eşyaları, Özel hava ve Manzara farklı katalog görünümleridir.

Satın alma

Bir ürünün fiyatı, görseli ve kimliği vardır. Satın alma sırasında bakiye ve sahiplik kontrol edilir; işlem kaydedilir. Zaten sahip olduğun parça için yeniden ücret alınmaması gerekir.

Yerleştirme

Sahip olunan bir dekor daha sonra bahçeye veya uygun odaya atanır. Aynı öğeyi yeni odaya taşımak, ikinci kopya üretmek değildir. Bu sürümde her katalog parçasının tek sahiplik kaydı bulunur.

Kaynak sayıları

Çalışan katalogta 186 DECOR kaydı var. Bu sayı yalnızca bahçe mobilyası demek değildir: ev eşyaları, tematik parçalar, iki saksı seçeneği ve üç hava modu da bu tabloda yer alır.

Hayvanlar ayrı

16 temel hayvan PETS tablosundadır. Gölet dekorunun içindeki küçük balık veya kurbağa, bağımsız satın alınan yol arkadaşı kaydıyla aynı şey değildir.

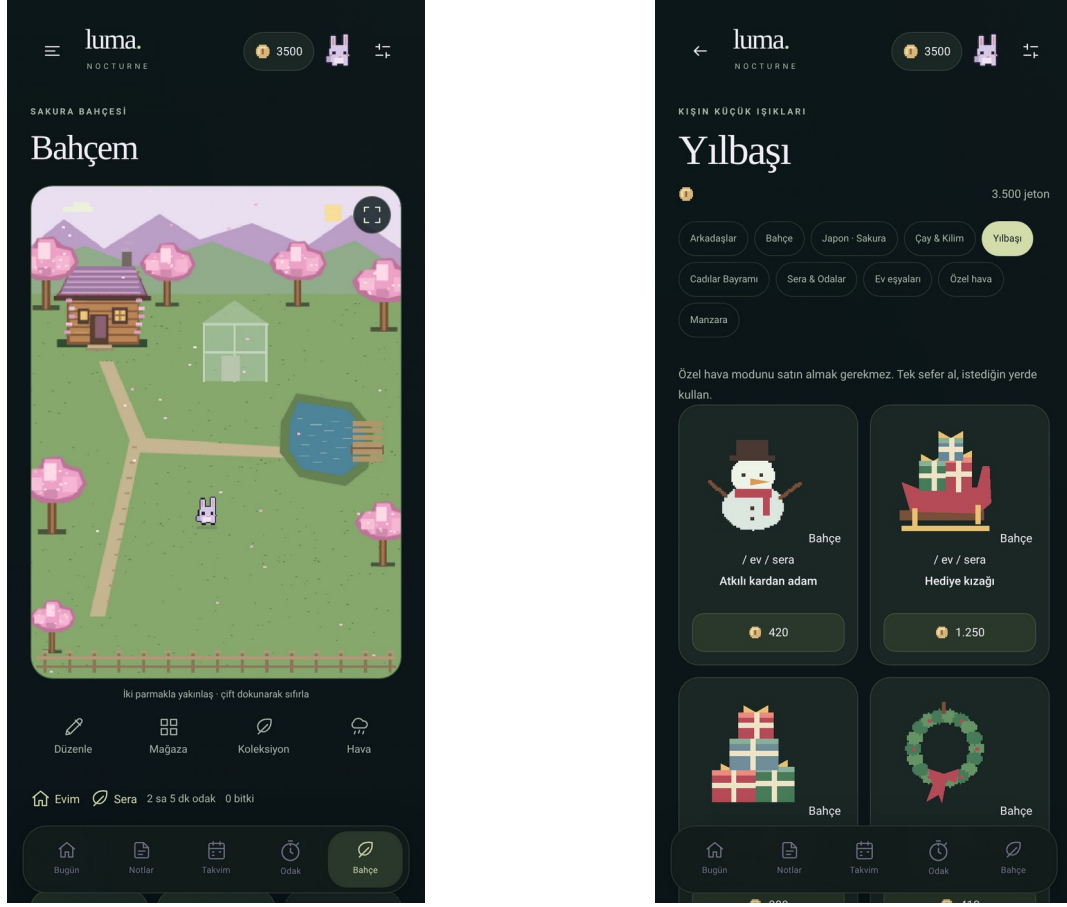
Öğrenme görevi

Ürün kimliği, satın alma kaydı ve sahne koordinatını üç ayrı kart gibi düşün. Birini değiştirmenin diğerlerini neden otomatik değiştirmemesi gerektiğini anlat.

Şekil 10. Japon · Sakura koleksiyonundan örnek mağaza görünümü.

Kaynak izi [K11, K14-K17] DECOR, PETS; buyItem; placeOwned; destinationSheet057. Sayılar koddan çalışma anında çıkarılmıştır.

Manzara, hava ve koleksiyon farkı



Şekil 11. Özel atmosfer sahneyi değiştirir; tematik dekor mağazası ayrı ürünler sunar.

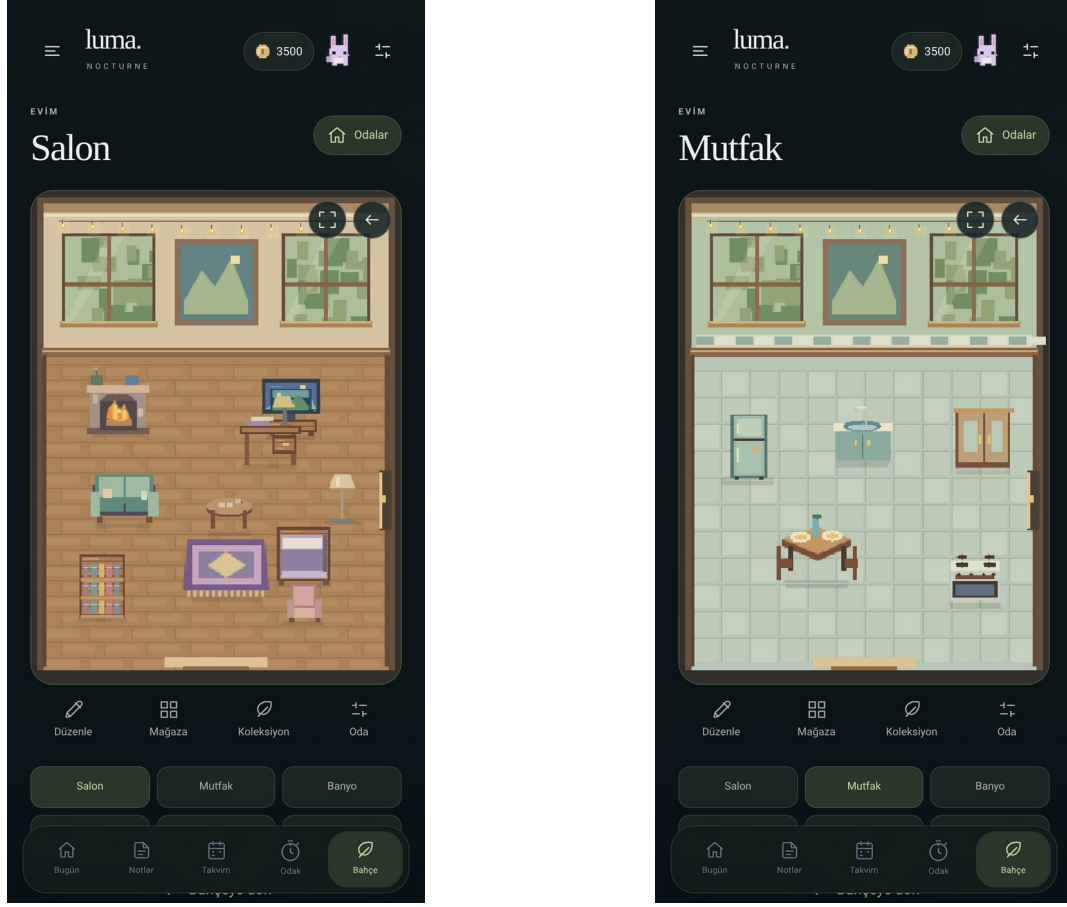
Manzara, bahçenin temel renk ve çevre profilidir. Ay Bahçesi başlangıçtan açıktır; Güneşli Çayırı 120, Yağmur Göleti 300, Sonbahar Korusu 600, Kar Vadisi 1.200 ve Yıldız Adası 2.400 kayıtlı çalışma dakikası gerektirir. Bunlar jeton fiyatı değil süre eşikleridir.

Özel hava modu satın alımı başka bir sistemdir: Sakura 4.500, Cadılar Bayramı 5.400, Yılbaşı 6.300 jetondur. Atmosfer; ağaç, zemin, süs ve parçacık görünümünü değiştirir. Normal hava seçimi Açık/Yağmur/Kar ve ışık seçimi Gündüz/Gün batımı/Gece olarak ayrıca saklanır.

Katalog dekorları ise tek tek alınıp yerleştirilen nesnelerdir. Sakura havasını açmak kırmızı kapı veya koi havuzunu otomatik vermez. Yılbaşı dekorunu kullanmak için Yılbaşı havasını satın almak da zorunlu değildir. Bu ayırım, kullanıcıya görsel özgürlük verirken veri modelini anlaşılır tutar.

Kaynak izi [K11, K14, K17] SCENERIES; ATMOSPHERES054; purchaseAtmosphere054; collectionTabs056.

Evim: birden fazla düzenlenebilir oda



Şekil 12. Salon ve mutfak. Yerleşimler öğretim için hazırlanmış örneklerdir.

Ev, tek bir küçük kutu yerine büyük dikey sahnelerden oluşur. Salon, mutfak, banyo, dinlenme odası ve yatak odası ayrı alan anahtarlarıyla tanımlanır. Odalar menüsünden geçiş yapılır. Eski ev içi kayıtlar uyumluluk için başlangıç salonu olarak yorumlanır.

Koleksiyondaki eşya hedef odaya atanır. Sahneye yerleşmiş eşyayı başka odaya taşırsan sahiplik korunur, hedef alan değişir; ikinci eşya yaratılmaz. Her oda için duvar ve döşeme renkleri ayrıca saklanır. Tek bir “ev rengi” değiştirmek bütün odaları zorunlu olarak aynı yapmaz.

Bir televizyonun dokununca açılıyor görünmesi, gerçek yayın hizmeti içerdiği anlamına gelmez. Şömine ve akvaryum gibi etkileşimler bu projede görsel durum değişimleridir. Kodda modellerin görünümü ile nesnenin ne işe yaradığı ayrı ayrı tanımlanmalıdır.

Sahiplik sınırı

Toplam sahne yerleşimi aynı world.layout listesini kullanır ve 180 öge sınırı vardır. Bu sınır sadece mutfak veya sadece salon için ayrı ayrı uygulanmış bir kapasite değildir.

Kaynak izi [K17] SPACES057; newLiving; itemSpace057; placeOwned; world.layout sınırı.

Sera: kişisel kış bahçesi



İçeri girme

Bahçe sahnesindeki seraya dokunarak veya Sera girişini kullanarak kış bahçesine açılırsın. Teknik olarak greenhouse anahtarıyla temsil edilen ayrı bir iç mekândır.

Sabit mimari

Cam çatı, ahşap taşıyıcılar ve bazı kenar ayrıntıları odanın çiziminden gelir. Bunlar satın alınmış kullanıcı mobilyalarıyla aynı kayıta tutulmaz.

Yerleştirilen eşyalar

Raf, sarmaşık, hasır koltuk, kanepeler, saksılama tezgâhı, tohum dolabı, teraryum, orkide, limon ağacı ve kitap sepeti katalogda ayrı ürünlerdir.

Katalog önerisi

Bir eşyanın recommendedRoom değeri satın alma akışına yardımcı olur. Bununla roomOnly kuralını karıştırma: önerilen yer ile yerleştirmeye izin verilen yer farklı kavramlardır.

Düzenleme

Seçili öğenin koordinatı değişir. Odanın kamera yaklaşması değişirse kayıtlı mobilya konumu değişmiş sayılmaz. Bu yüzden yerleşim, ekran pikseli olarak değil sahne bilgisi olarak tutulur.

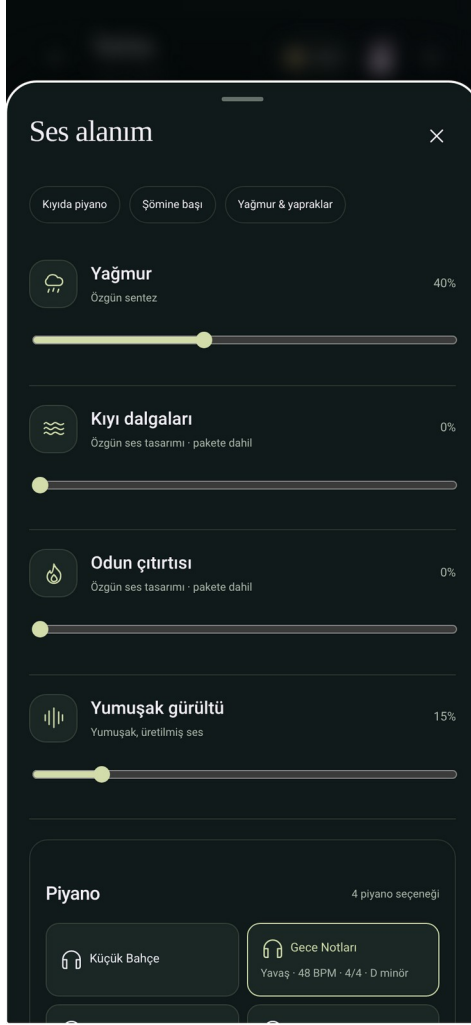
Öğrenme görevi

Bir koltuğu seraya yerleştir, sonra salona taşı. İki alanda aynı anda görünmediğini ve yeniden satın alma gerekmediğini kontrol et. Tek kimlikli kayıt ilişkisini gözlemler.

Şekil 13. Kış bahçesi; büyük dikey sahnede farklı eşya ve bitki modelleri.

Kaynak izi [K17] haven-model.js → SPACES057; haven-garden.js → HomeGarden057; haven-shell.js → destinationSheet057.

Ses alanım: birden fazla kanalı karıştırmak



Doğa kanalları

Yağmur, kıyı dalgaları, odun çıtırtısı ve yumuşak gürültü gibi kanalların seviyeleri bağımsızdır. Bir kaydırıcı tek bir genel ses yerine ilgili kanalın yüzdesini değiştirir.

Piyano seçimi

Küçük Bahçe seçeneğine ek olarak Gece Notları, Sabah Valsi ve Camdaki Damlalar bulunur. Seçim, aynı anda açık piyano kanallarını denetleyen bir arayüz mantığıyla yönetilir.

Kaynak türü

Paketteki Atölye okyanus/ateş sesleri ve yeni piyanolar sentezle üretilmiştir. Gerçek kıyı kaydı veya gerçek akustik piyano performansı olarak anlatılmamalıdır.

Hazır karışım

Kıyıda piyano, Şömine başı, Yağmur & yapraklar gibi düzenler ses kanallarına başlangıç yüzdeleri verir. Ses dosyalarının birleştirilip tek yeni dosya yazılması gerekmez.

Telefon davranışı

AppState etkin değilken ses oynatma kapatılır. AudioEngine de arka plan çalmayı false olarak ayarlar. Bu sürümde ekran kilitlenince devam eden müzik özelliği varmış gibi düşünme.

Öğrenme görevi

Bütün kanalları sıfırla, bir doğa kanalını aç, sonra düşük piyano ekle. Bir kanalın seçilmesi ile genel playing bayrağının açık olması arasındaki farkı gözlemle.

Şekil 14. Ses katmanları. Kaynak açıklaması ve kanal yüzdeleri birbirinden ayrıdır.

Kaynak izi [K02, K22, K27] AudioEngine.tsx; haven-audio.js; atelier-audio.js; Audio-Included.json.

Haftam: kayıtları anlamlı bir özete çevirmek



Ölçülen şey

Haftalık görünüm kayıtlı odak oturumlarını toplar. Bu, telefonun açık kaldığı süre veya bütün takvim etkinliklerinin süresi değildir.

Özet kartları

Dakika, oturum ve aktif gün sayıları aynı geçmişin farklı hesaplamalarıdır. Haftayı değiştirdiğinde yeni veri kopyası oluşturulmaz; aynı kayıtlar farklı tarih aralığında süzülür.

Günlük grafik

Her çubuk o günün toplam çalışma süresini gösterir. Grafikteki örnek değerler demo oturumlarından gelir. Görsel uzunluğun arkasında sayısal toplam vardır.

Kategori dağılımı

Bir oturumun bağlı olduğu not üzerinden kategori ilişkisi kurulabilir. Kayıt başka ekrana bağlı olduğu için kimliklerin tutarlı kalması önemlidir.

Hedef ve düşünce

İsteğe bağlı hafta hedefi, niyet ve değerlendirme ayrı alanlarda tutulur. Değerlendirmeyi nota dönüştürmek mevcut notları değiştirmek yerine yeni bir not oluşturabilir.

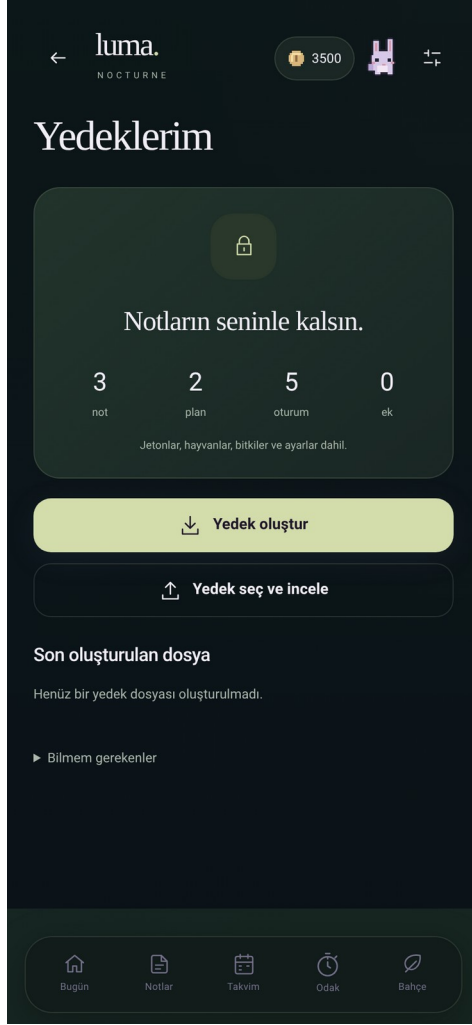
Öğrenme görevi

Aynı gün iki kısa oturum ve başka gün bir oturum düşün. Oturum sayısı 3, aktif gün sayısı 2 olur. Bir istatistiğin neyi saydığını açıkça tanımlamak, güzel grafik çizmekten önce gelir.

Şekil 15. Haftalık özet; gün bazlı çubuklar ve kategori toplamaları.

Kaynak izi [K20] weekly.js → mondayFor, weeklyStats, renderWeekly, weeklyToNote.

Yedeklerim: dosya oluşturmak ve geri yüklemek



Neler bulunur?

Yedek, notlar, planlar, odak geçmişi, jeton/alışveriş ve bahçe gibi uygulama verilerini kapsar. Ulaşılabilen dosya ekleri de dışa aktarma paketine gömülür.

Oluşturmak ve saklamak

Yedek oluşturmak önce uygulamanın alanında dosya üretir. Paylaş / Dosyalar'a kaydet ile onu seçtiğin yere ayrıca saklarsın. Kod ZIP'i bunun yerine geçmez.

Parola seçeneği

Parola seçildiğinde dışa aktarılan dosya şifrelenir. Parola uygulama kaydına yazılmaz. Bu, telefonun canlı state.json dosyasının da şifrelendiği anlamına gelmez.

Önizleme

Geri yükleme dosyasını seçmek hemen bütün veriyi değiştirmez. Önce içerik sayıları gösterilir; gerekiyorsa parola açılır. Onaylanan aday veri sonra normalize edilip kaydedilir.

Geri yüklemenin sonucu

Mevcut kayıtların yerine yeni yedek geçer; iki ayrı veri grubunun otomatik birleştirilmesi yapılmaz. Etkin sayaç ve bildirimler bu geçişte ayrıca ele alınır.

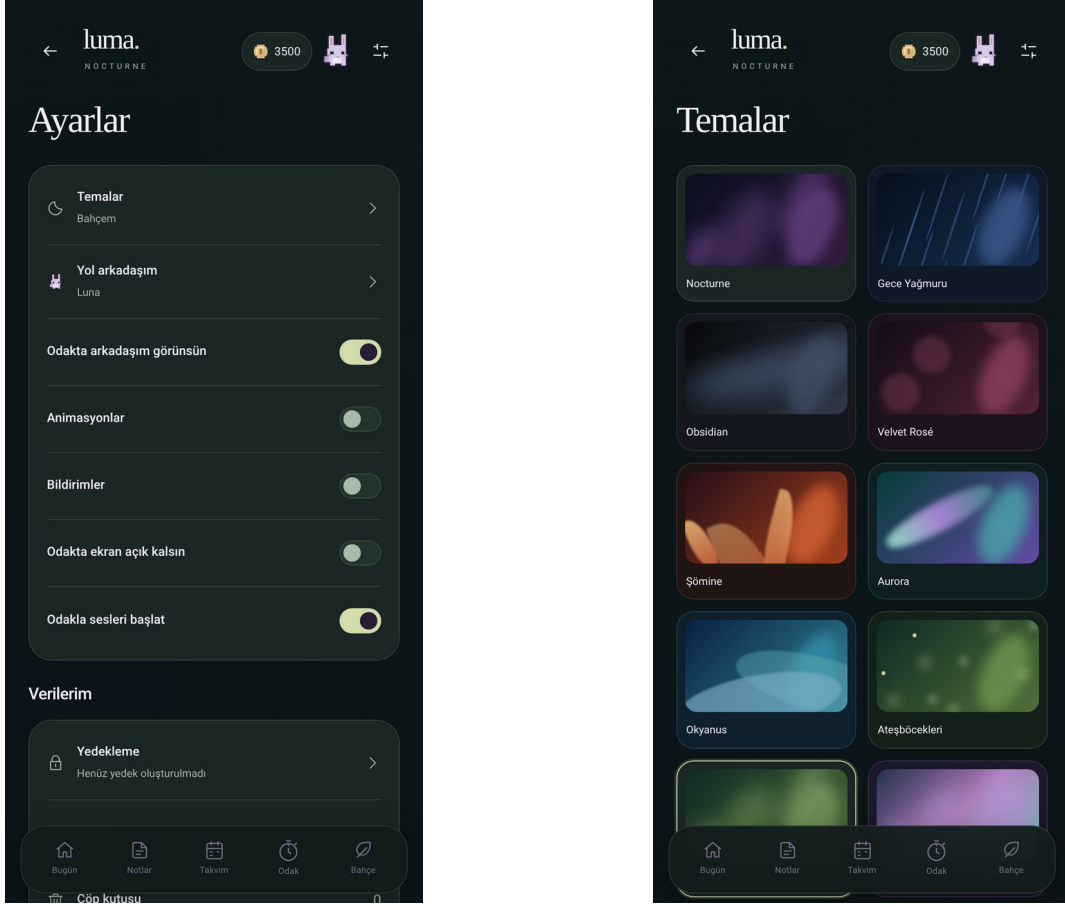
Öğrenme görevi

Gerçek tek veri kopyanda deneme yapma. Öğrenme örneğinde "önizleme durumu" ile "kayıdı değiştirme durumu" için neden iki ayrı aşama gerektiğini açıkla.

Şekil 16. Yedeklerim; örnek veri sayıları gösterilir, dosya henüz oluşturulmamıştır.

Kaynak izi [K05, K06, K21] NativeServices.ts; BackupCrypto.ts; backup.js.

Ayarlar, temalar ve ortak menüler



Şekil 17. Ayarlar ve görünüm seçimi. Tema değiştirmek veri sıfırlamak değildir.

Ayarlar, uygulamanın kişiye göre davranışını düzenler. Hareketli arka plan, yol arkadaşının görünmesi, odak sırasında ekranı açık tutma, bildirim izni ve sesin çalışmayla ilişkisi tercih alanlarıdır. Bir tercih true/false değeri olabilir; tema gibi başka tercihler kayıtlı seçenek kimliği tutar.

Son pakette 13 uygulama teması tanımı vardır: Nocturne, Gece Yağmuru, Obsidian, Velvet Rosé, Şömine, Aurora, Okyanus, Ateşböcekleri, Bahçem, Opal Akışı, Zümrüt Cam, Kehribar Gece ve Kozmik İnci. Her tema sadece bir renk değildir; effect alanı farklı çiziciye veya animasyona yol açabilir.

Alt menüler ortak sheet işlevinden doğar. Başlık, içerik, kapatma ve sürükleme davranışı aynı yapı üzerinden paylaşılır. Bu ortaklık, bir menü düzeltmesinin diğerlerini de etkilemesini sağlar; aynı zamanda değişiklik yaparken bütün menü türlerini test etmeyi gerektirir.

Uygulama teması ≠ bahçe atmosferi

Nocturne veya Opal gibi tema bütün arayüzün görünümünü etkiler. Sakura veya Yılbaşı atmosferi ise bahçenin özel süs ve çevre düzenidir. Bu iki seçim kodda aynı alan değildir.

Kaynak izi [K03, K14, K18] themes ve defaultPrefs; ATMOSPHERES054; stable-shell.js; haven-shell.js.

Hangi dil, hangi işi yapıyor?

Bir uygulama “tek bir dille yazılmış” olmak zorunda değildir. LumaNote’ta kullanıcıya görünen sayfalar, telefonla konuşan katman ve geliştirme araçları farklı görevler üstlenir. Aşağıdaki sınıflandırma, dosya uzantılarına bakınca ne gördüğünü anlamayı sağlar.

Teknoloji	Bu projedeki görevi	Örnek dosya
JavaScript · .js	Sayfalar, olaylar, sayaç, alışveriş ve sahne davranışları.	tools/ui/stable-focus.js
TypeScript · .ts / .tsx	Telefon kabuğu, tipler, dosya/ses/bildirim servisleri.	App.tsx, NativeServices.ts
HTML	Sayfanın başlık, düğme, liste ve alanlarını tarif eden işaretleme.	tools/ui/index.html
CSS	Renk, boyut, yerleşim, katman ve bazı animasyonlar.	tools/ui/haven.css
GLSL	Lava görünümü için ekran kartında çalışan shader kodu.	tools/ui/lava.js
Python · .py	Ses üretimi ve tarayıcı testleri gibi geliştirme işleri.	tools/ ve tests/ altındaki betikler
Bash · .command	Mac Terminal’inde başlatma ve güncelleme adımları.	BASLAT.command
JSON	Ayarları ve kayıtları metin biçiminde taşıyan veri formatı.	package.json, state.json

Dil olmayan ama sık duyacağın isimler

React bir kullanıcı arayüzü kütüphanesidir. React Native, React yaklaşımıyla telefon bileşenleri kullanan çerçevedir. Expo geliştirme araçları ve hazır telefon modülleri sağlar. WebView uygulamanın içine yerleştirilmiş web görünümüdür. Node.js, JavaScript’i Terminal tarafında çalıştıran ortamdır. npm paket yönetir; Metro uygulama kodunu paketler. Bunların her biri yeni bir programlama dili değildir.

Projene özel önemli sonuç

HTML/CSS öğrenmek burada yalnızca “web sitesi” yapmak için değildir. LumaNote’un gerçek ekranlarının büyük bölümü bu dillerle tanımlanır. TypeScript tarafı ise o ekranı telefonun dosya, ses ve bildirim özelliklerine bağlar.

Kaynak izi [K01–K04, K14, K28] Paket, kaynak ve araç envanteri. Genel kavramlar: [D01–D05, D07].

HTML iskeleti, CSS görünümü kurar

HTML bir öğenin ne olduğunu söyler: başlık mı, düğme mi, metin kutusu mu? CSS aynı öğenin nasıl görüneceğini söyler: rengi, boşluğu, büyüklüğü ve ekrandaki konumu. JavaScript ise kullanıcı o öğeyle etkileşince ne olacağını belirler.

Öğrenme örneği · uygulamaya otomatik eklenmez

```
<section class="kart">
  <h2>Bugünkü hedefim</h2>
  <p>Bir küçük adımla başla.</p>
  <button id="basla" type="button">Başla</button>
</section>
```

section bir bölümü gruplar. h2 başlıktır. p paragraftır. button gerçek bir düğmedir. class aynı stile sahip öğeleri toplar; id belirli bir öğeyi bulmaya yarar. İçerik ve etiket farklı şeylerdir: “Başla” yazısını değiştirmek, düğmenin davranışını kendiliğinden değiştirmez.

Öğrenme örneği · uygulamaya otomatik eklenmez

```
.kart {
  background: #172523;
  color: #f3ecfa;
  padding: 24px;
  border-radius: 20px;
}
.kart button { min-height: 44px; }
```

CSS’te seçicinin ardından süslü parantezler içinde özellik: değer çiftleri yazılır. padding iç boşluktur; margin dış boşluktur. border-radius köşeleri yuvarlar. Bu örnekte .kart button, kartın içindeki düğmeleri seçer. Düğmeye class="kart" vermekle aynı anlamda değildir.

LumaNote’ta nereye bakmalısın?

tools/ui/index.html ana yerleştirme alanlarını barındırır. Ekran parçalarının çoğu JavaScript işlevlerinden HTML metni olarak üretilir. Görünüm style.css ile başlar; sürüm katmanlarının CSS dosyaları sonradan yüklenir. Aynı kuralın daha sonra gelmesi önceki görünümü değiştirebilir. Bu yüzden “renk kodunu değiştirdim ama görünmedi” sorununda yalnızca ilk CSS dosyasına bakmak yetmez.

Kaynak izi [K03, K04, K07] index.html; style.css; screens.js; build-ui.cjs.

JavaScript: değer, koşul ve işlev

Kod, değerler üzerinde açık kurallarla işlem yapar. Bir değişken bir değere isim verir. string metindir; number sayıdır; boolean true veya false durumudur. null ise bilerek boş bırakılmış değeri temsil edebilir. "25" metni ile 25 sayısı aynı tür değildir.

Öğrenme örneği · uygulamaya otomatik eklenmez

```
const hayvanAdi = "Luna";
let dakika = 25;
const molaMi = false;

function jetonHesapla(saniye) {
  if (!Number.isFinite(saniye) || saniye < 0) {
    throw new Error("Geçerli süre gerekli.");
  }
  return Math.floor(saniye / 20);
}

const sonuc = jetonHesapla(dakika * 60); // 75
```

const değişkenin başka bir değere yeniden atanmasını engeller. let yeniden atanabilir. function bir işi isimlendirir; parantez içindeki saniye girdidir. return sonucu dışarı verir. if koşula göre yol seçer. throw işlemin normal sonuç veremediğini bildirir. Math.floor küsuratı aşağı yuvarlar.

Satırları zihninde çalıştır

dakika * 60, 25 dakikayı 1.500 saniyeye çevirir. $1.500 / 20 = 75$ olur. Fonksiyon 75 döndürür ve sonuc değişkeni bunu tutar. Gerçek LumaNote bakiyesi bundan daha fazlasını yapar: eski sürüm geçişini, toplam süreyi, alışverişleri ve test bütçesini ayrıca hesaba katar. Bu küçük fonksiyon gerçek cüzdanın yerine konmamalıdır.

Hata yapmak öğrenmenin parçası

Bir düğme çalışmıyorsa önce beklediğin girdiyi yazdır: console.log(saniye). Ardından işlemden çıkan sonucu kontrol et. Değerleri görmeden rastgele kod değiştirmek yerine hangi varsayımın bozulduğunu ara.

Mini alıştırma: jetonHesapla(19), jetonHesapla(20) ve jetonHesapla(59) için sonuçları önce kâğıda yaz. Ardından tarayıcı konsolunda dene. Cevaplar sırasıyla 0, 1 ve 2 olmalıdır.

Kaynak izi [K10, K11] Uygulamadaki süre/jeton ilişkisine dayalı öğretim örneği; gerçek wallet hesabı ayrıca açıklanır.

Nesne, dizi, kimlik ve koleksiyon

Bir not yalnızca tek bir yazı değildir; başlığı, kimliği ve durumu vardır. Bu alanları bir object yani nesne gruplar. Birden fazla not ise array yani dizi içinde tutulur. Kimlik, başlık değişse bile aynı notu bulabilmek için kullanılır.

Öğrenme örneği · uygulamaya otomatik eklenmez

```
const notlar = [
  { id: "n1", title: "Ders", deletedAt: null },
  { id: "n2", title: "Alışveriş", deletedAt: "2026-10-01" }
];
const aktifler = notlar.filter(n => !n.deletedAt);
const basliklar = aktifler.map(n => n.title);
const secili = new Set(["n1", "n1"]);
console.log(secili.size); // 1
```

filter koşula uyanları seçer; map her öğeden başka bir değer üretir. `n => n.title` kısa bir işlev yazımıdır. Set aynı değeri yalnızca bir kez tutar; toplu not seçiminde aynı kimliği iki kez saymamak için uygundur. Map anahtar ile değeri eşleştirir; LumaNote köprüsünde mesaj kimliğinin yanıtını bekleyen kaydı bulmak için kullanılır.

İfade	Anlamı
<code>note.title</code>	Nesnenin title alanını oku.
<code>note?.title</code>	note yoksa doğrudan alan erişimi hatası üretmeden ilerle.
<code>value varsayılan</code>	İlk değer falsy ise diğerini kullan; 0 ve boş metin de etkilenir.
<code>value ?? varsayılan</code>	Yalnızca null veya undefined ise diğerini kullan.
<code>{ ...note, title: "Yeni" }</code>	Alanları kopyalayıp başlığı değiştiren yeni bir nesne oluştur.
<code>JSON.stringify / JSON.parse</code>	Veriyi JSON metnine çevir / JSON metninden veri oku.

Kopya her zaman derin kopya değildir

`{ ...nesne }` yalnızca ilk düzeyi kopyalar. İç içe bir diziyi değiştirirsen iki yerde aynı diziyi kullanıyor olabilirsin. LumaNote birçok kayıt/geri alma noktasında JSON ile anlık kopya alır; bu tekniğin yalnızca JSON'a uygun değerleri taşıdığını unutma.

Kaynak izi [K03, K08, K11] `core.js`; `polish-notes.js`; `world.js` içindeki veri koleksiyonları.

Olaylar ve beklenen işlemler

Kullanıcı düğmeye bastığında bir olay oluşur. JavaScript bir olay dinleyicisiyle bunu karşılar. Bazı işler anında biter; dosyaya yazma gibi işler daha sonra sonuç verir. Promise, ileride tamamlanacak bu sonucu temsil eder. `async/await`, bu akışı okunur biçimde yazmayı sağlar.

Öğrenme örneği · uygulamaya otomatik eklenmez

```
let mesgul = false;
async function kaydet() {
  if (mesgul) return;
  mesgul = true;
  try {
    await dosyayaYaz(); // Öğretim amaçlı varsayılan işlev
    console.log("Kayıt tamamlandı.");
  } catch (hata) {
    console.error("Kaydedilemedi:", hata);
  } finally {
    mesgul = false;
  }
}
```

`await`, o işlemin devamını sonuç gelene kadar bekletir; bütün arayüzü otomatik olarak dondurması gerekmez. `catch` reddedilen veya hata atan yolu karşılar. `finally` her iki durumda da çalışır. `mesgul` bayrağı kullanıcı art arda bastığında aynı işlemi iki kez başlatmayı önler. Bayrak hata yolunda sıfırlanmazsa düğme sonsuza kadar çalışmıyor gibi görünebilir.

Olay delegasyonu ne demek?

LumaNote'ta birçok HTML düğmesinde `data-action` bulunur. Ortak olay yönlendiricisi tıklanan öğeye bakar ve ilgili işleve gider. Böylece her ekran tekrar çizildiğinde her düğmeye ayrı dinleyici bağlamak gerekmez. Ama aynı `action` adını iki farklı anlam için kullanırsan karışıklık oluşur.

Kritik iş ve isteğe bağlı iş

Bir çalışmanın kaydı kritik; kısa bir bildirim sesi isteğe bağlıdır. Kayıt başarısızken "jeton kazandın" dememek gerekir. Buna karşılık ses modülünün yanıt vermemesi, duraklat düğmesini kilitlememelidir. `stable-focus.js` bu ayrımı yapmaya çalışır.

Kaynak izi [K03, K10] `events.js`; `core.js` → `Native.call/persist`; `stable-focus.js`.

TypeScript: verinin biçimini tarif etmek

TypeScript, JavaScript'e tür bilgisi ekler. Kod yazarken “bu alan sayı olmalı”, “bu mesajın type alanı olmalı” gibi beklentileri denetlemeye yardımcı olur. Türler telefon ekranındaki yeni bir özellik değildir; geliştiricinin yanlış eşleşmeleri daha erken görmesine yarar. [D03]

Gerçek kaynak alıntısı · nocturne/types.ts · satır düzeni açıldı

```
export type AudioSettings = {
  playing: boolean;
  volumes: Record<string, number>;
  customAudio?: Attachment | null;
};
```

playing açık/kapalıdır. Record<string, number>, rain veya ocean gibi metin anahtarlarının sayısal değerleri olduğunu söyler. ? işareti customAudio alanının bulunmayabileceğini anlatır. Attachment | null, alan varsa bir dosya nesnesi veya null olabileceği anlamına gelir.

Derleme kontrolü ile gerçek veri kontrolü farklıdır

TypeScript türü yazmak, içe aktarılan bir JSON dosyasını kendiliğinden güvenli yapmaz. Dosya dışarıdan geldiğinde çalışma anında kontrol gerekir. Projede isAppData temel alanların dizi olup olmadığına bakar; normalize işlevleri daha ayrıntılı temizleme ve varsayılan tamamlama yapar.

Öğrenme örneği · uygulamaya otomatik eklenmez

```
type KisaNot = { id: string; title: string };
const not: KisaNot = { id: "n1", title: "Merhaba" };
// not.title = 42; // Tür kontrolünde hata beklenir.
```

Projedeki kapsamı doğru oku

App.tsx ve native servisler TypeScript kullanır. Buna karşılık tools/ui altındaki işlevlerin çoğu düz JavaScript'tir. AppData içinde unknown ve açık alanlar bulunur. Dolayısıyla “bütün proje tam ve katı tip güvenli” demek doğru olmaz.

Kaynak izi [K02, K05, K06] nocturne/types.ts; App.tsx; normalize akışları. Genel tanım: [D03].

React ve telefon kabuğu

React bileşeni, bir arayüz parçasının görünümünü ve davranışını gruplar. React Native'de View ve Text gibi bileşenler telefon katmanında kullanılır. LumaNote'un ayrımı şudur: native React bileşeni bütün Notlar veya Bahçe sayfasını tek tek çizmez; bu sayfaları taşıyan WebView'i ve telefon servislerini kurar. [D04]

Gerçek kaynak alıntısı · App.tsx · ilgili satırlar düzenlenerek gösterildi

```
const web = useRef<WebView>(null);
const [audio, setAudio] = useState<AudioSettings>(INITIAL_AUDIO);
const services = useMemo(() => createNativeServices({
  emit: (e: NativeEvent) => sendRef.current(e),
  setAudio,
  setAppearance
}), []);
```

Yapı	Ne için kullanılır?	Bu projedeki örneği
useState	Değiştğinde React görünümünü güncelleyen durum.	Ses ayarları, tam ekran, hata ekranı.
useRef	Çizimler arasında korunan bir referans.	WebView'e mesaj göndermek.
useMemo	Hesaplanan değeri bağımlılıklara göre yeniden kullanmak.	Native servis nesnesini yeniden kurmamak.
useEffect	Bileşen yaşamında yan iş ve temizlik.	AppState ve geri tuşu dinleyicileri.

Ekran kapandığında dinleyicileri kaldırmak, sesleri durdurmak ve sahne çizicisini bırakmak önemlidir. Bir olayın her sayfa açılışında yeniden bağlanıp hiç temizlenmemesi, tek dokunuşun birkaç kez çalışmasına neden olabilir. Projedeki useEffect dönüşlerindeki temizleme kodunu bu gözle incele.

JSX nedir?

App.tsx içindeki `<WebView ... />` yazımı JSX'tir. HTML'ye benzer görünür ama React bileşenlerini oluşturur. tools/ui içindeki HTML metniyle aynı çalıştırma yolu değildir. İki yerdeki düğmenin dosya ve olay biçimi de farklı olabilir.

Kaynak izi [K02] App.tsx → Application, onMessage, App. React Native kavramı: [D04].

Geliştirme araçlarının görevleri

Araç / uzantı	Günlük benzetme	LumaNote'ta gerçek iş
Node.js	Telefon dışında çalışan atölye.	Arayüzü birleştirir, doğrular, kurulum betiklerini yürütür.
npm	Hazır parçaların listesi ve kurucusu.	package.json'daki paketleri kurar, scripts komutlarını çalıştırır.
Metro	Paketleyici.	Expo'nun çalıştıracağı JavaScript bağımlılıklarını bir araya getirir.
Babel	Dönüştürücü.	Kaynak kodu seçilen Expo ortamının beklediği biçime dönüştürür.
Python	Üretim ve test tezgâhı.	Ses verisi üretir, Playwright ve dosya testlerini çalıştırır.
Bash	İş sırası listesi.	Kontrol, yedek, Node çağrısı ve başlatma adımlarını sıraya koyar.
FFmpeg	Ses işleme aracı.	Üretilen sesi MP3'e dönüştürür; testte dosyayı yeniden çözer.

Paket dosyaları nasıl okunur?

package.json istenen bağımlılıkları ve komutları listeler. package-lock.json varsa çözümlenen paket ağacını daha kesin kaydeder. node_modules gerçek kurulu paket klasörüdür; uygulamanın kendi özgün kaynak koduyla aynı şey değildir. Paket kilidinin görevi npm belgelerinde de bu tekrar üretilebilirlik amacıyla açıklanır. [D06]

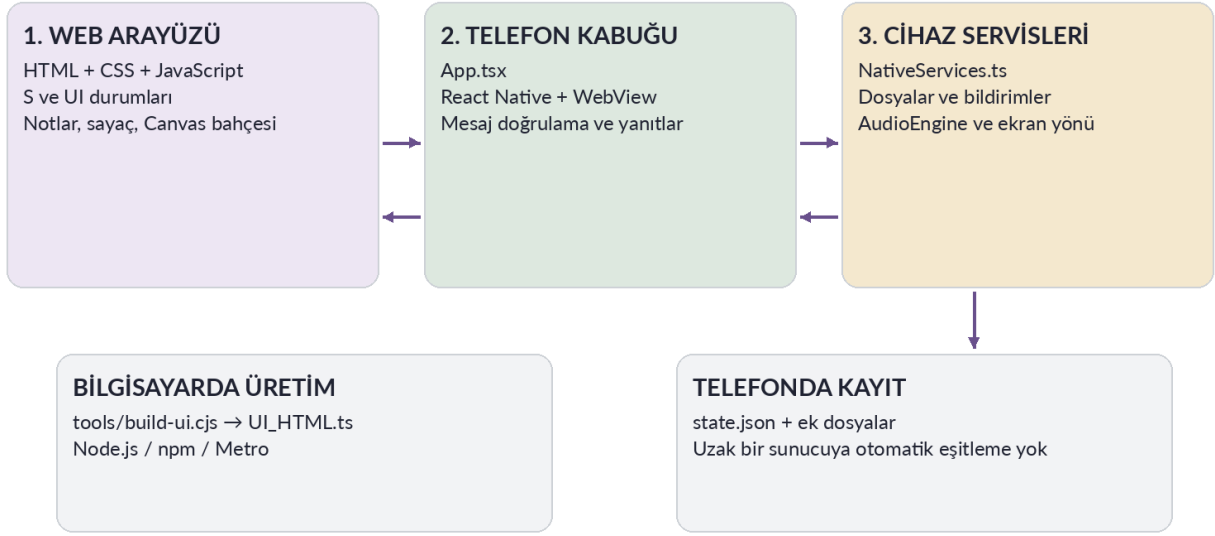
Bu arşiv Expo için ~57.0.26, React için 19.2.3 ve React Native için 0.86.3 beyan eder. Node motor aralığı ≥ 24 < 25 'tir. Bunlar son paket içindeki hedeflerdir; Mac'indeki o an kurulu paket ağacını uzaktan ölçtüğümüz anlamına gelmez. İnternetteki "en yeni sürüm" ile projedeki sürümü gelişigüzel karıştırma.

Programlama öğrenirken hangi sırayla?

Önce HTML/CSS ve küçük JavaScript işlevleri. Sonra olaylar, diziler ve async/await. Ardından TypeScript ve React kabuğu. Shader, ses sentezi ve şifreleme uygulamanın daha ileri alanlarıdır; ilk not düğmeni yapmak için bunların tamamını bilmen gerekmez.

Kaynak izi [K01, K04, K24, K28] package.json; build-ui.cjs; setup.cjs; üretim/test betikleri. [D06, D07].

Bir uygulama, iki çalışma alanı



Şema 2. Telefon içindeki iki katman ve geliştirme sırasında kaynak üretimi.

WebView tarafı ekrandaki yazıyı, düğmeleri, bahçe canvas'ını ve uygulama durumunu yönetir. Native taraf telefon dosyasına yazma, belge seçme, bildirim planlama, ses oynatma ve ekran yönü gibi işleri üstlenir. İki taraf aynı JavaScript değişkenini doğrudan paylaşmaz; mesaj gönderir.

Neden bu ayrım önemli?

Bir not başlığı eklemek çoğunlukla web arayüzü işidir. Fotoğraf galerisini açmak ise telefondaki izne ve seçiciye erişim ister. Bahçe ağacını çizmek Canvas işidir; telefonu yatay kilitlemek ScreenOrientation işidir. Sorunun hangi katmanda olduğunu bulursan doğru dosyaya daha hızlı gidersin.

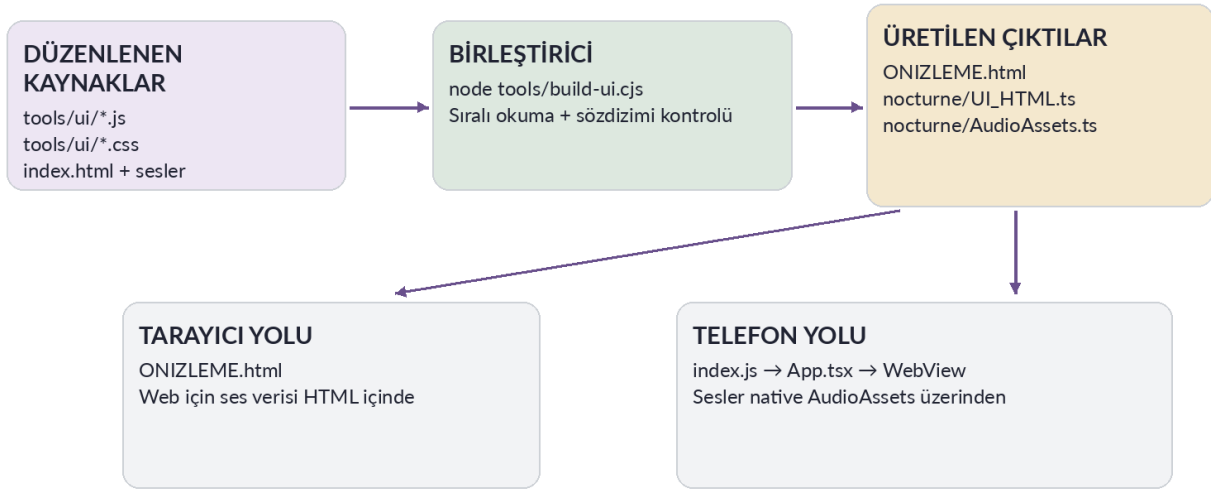
ONİZLEME.html tarayıcıda çalıştığında gerçek native köprü yoktur; browser.js karşılıkları devreye girer. Bu, arayüzü hızlı görmeyi sağlar. Ama bir tarayıcı ekranının görünmesi, iPhone dosya seçicisinin, klavyesinin veya bildirimlerinin de test edildiğini göstermez.

Bu projede bulunmayan katman

Uzak API sunucusu, oturum açılan kullanıcı veritabanı ve cihazlar arası senkronizasyon servisi bu paket içinde yoktur. “Backend” sözcüğünü burada NativeServices ile eş anlamlı kullanma: NativeServices telefonun içinde çalışır.

Kaynak izi [K02–K05, K23] App.tsx; core.js Native; NativeServices.ts; browser.js. WebView mesajlaşma kavramı: [D02].

Kaynak nasıl telefon ekranına dönüşür?



Şema 3. Tek arayüz kaynağından iki farklı çıktı. Üretilmiş dosyalar elle düzenleme noktası değildir.

tools/build-ui.cjs, index.html içindeki CSS ve JavaScript yer tutucularını gerçek kaynaklarla değiştirir. CSS ve JS dosyalarının sırası açık bir listede tanımlıdır. Birleştirilen JavaScript önce sözdizimi denetiminden geçer; sonrasında tarayıcı önizlemesi ve native WebView metni üretilir.

Tarayıcı önizlemesi ses verilerini içine gömebilir. Native sürümde sesler AudioAssets.ts içindeki statik dosya başvurularıyla Expo Audio'ya verilir. PDF kitaplıkları kuruluysa bunlar da paketlenir; kurulu değilse kod bunun eksikliğini bildirir. Dolayısıyla üretilen dosyanın büyüklüğü veya içeriği kurulu varlıklara göre farklılaşabilir.

Mevcut proje klasöründe kullanılan gerçek komutlar

```
npm run build:ui
npm run verify
```

En sık yapılan düzenleme hatası

Yalnızca ONIZLEME.html veya nocturne/UI_HTML.ts üzerinde değişiklik yaparsan sonraki build:ui bunu ezer. Değişiklik tools/ui içindeki kaynakta yapılmalıdır. Yeni dosya eklemek de yeterli değildir; gerekiyorsa build listesine dahil edilmelidir.

Kaynak izi [K01, K04] tools/build-ui.cjs; package.json scripts; nocturne/AudioAssets.ts.

Dosya ağacında kaybolmamak

Kaynak arşivin sadeleştirilmiş dizin haritası

```
LumaNote_057_Tam_Proje/
├── index.js                # Expo giriş noktası
├── App.tsx                 # Telefon kabuğu
├── app.json / package.json # Kimlik, tercihler, paketler
├── nocturne/
│   ├── NativeServices.ts   # Dosya, bildirim, paylaşım
│   ├── AudioEngine.tsx    # Native ses oynatma
│   ├── BackupCrypto.ts    # Şifreli yedek biçimi
│   ├── types.ts           # TypeScript veri tarifleri
│   └── UI_HTML.ts         # ÜRETİLMİŞ arayüz
├── tools/
│   ├── ui/                # Asıl ekran ve davranış kodu
│   ├── build-ui.cjs       # Arayüz derleme betiği
│   ├── setup.cjs         # Kurulum / başlatma
│   └── repair-existing.cjs # Yedekli yerinde güncelleme
├── assets/                # Ses ve görsel varlıkları
├── licenses/              # Varlık kaynak / lisans bilgisi
├── tests/                 # Test kaynakları ve sonuçları
├── docs/                  # Sürüm raporları ve görseller
└── ONIZLEME.html         # ÜRETİLMİŞ tarayıcı görünümü
```

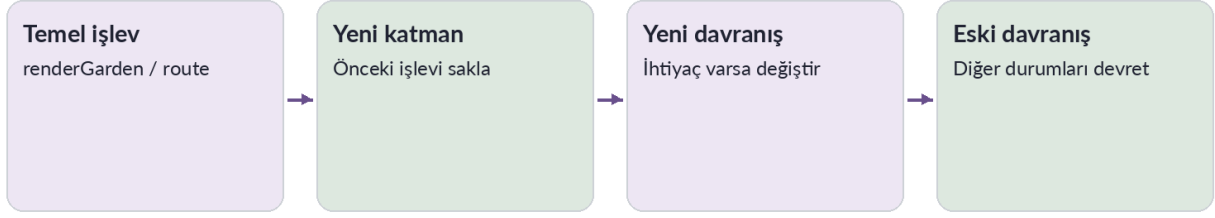
Bir değişikliğin adresini bul

Değişiklik	İlk bakılacak yer
Not listesinin yerleşimi	living-notes.js, polish-notes.js, ilgili CSS
Sayaç kaydı / ödül	stable-focus.js, living-model.js
Bahçe hareketi / odalar	atelier-garden.js, haven-garden.js
Yeni ürün / model	...-model.js veya koleksiyon dosyası; ...-art.js
Telefonda dosya okuma	NativeServices.ts
Yeni ses seçeneği	...-audio.js, AudioEngine.tsx, build-ui.cjs, ses varlıkları

Bu tablo tek dosyada her şey biter demek değildir. Bir özelliğin veri modeli, düğmesi, çizimi, kayıt temizleyicisi ve testi farklı dosyalarda olabilir. Kaynak haritası bölümünde ilgili işlev adları ayrıca listelenir.

Kaynak izi [K01–K29] Son arşivin gerçek dosya ağacı; sadeleştirilmiş gösterim.

Eski işlevler neden tekrar tanımlanmış?



Dosya adı kadar yükleme sırası da önemlidir: son tanım, çalışma anındaki davranışı etkiler.

Şema 4. Yükleme sırası, son kullanılan davranışı belirler. Bu şema bütün dosya listesinin yerine geçmez.

LumaNote birçok sürüm boyunca genişletildi. Bu yüzden tools/ui altında temel kodların ardından world, living, stable, polish, sakura, atelier ve haven gibi katmanlar görürsün. Yeni katman bazı işlevlerin önceki sürümünü bir değişkende saklayıp yeni davranışla sarar. Bazı işlevleri ise doğrudan yeniden atar.

Sarma yaklaşımını anlatan sadeleştirilmiş örnek

```

const oncekiRender = renderGarden;
renderGarden = function () {
  const html = oncekiRender();
  return html + "<p>Yeni bölüm</p>";
};
  
```

Örnekte ilk işlev kaybolmaz; yenisi onu çağırır ve sonuca içerik ekler. Gerçek kaynakta daha ayrıntılı veri geçişleri, görsel değişiklikler ve sınıf türetmeleri vardır. Bir işlevin dosyada ilk geçtiği yeri okuyup “uygulama kesin bunu kullanıyor” sonucuna varma. Daha sonra yeniden atanıp atanmadığını ara.

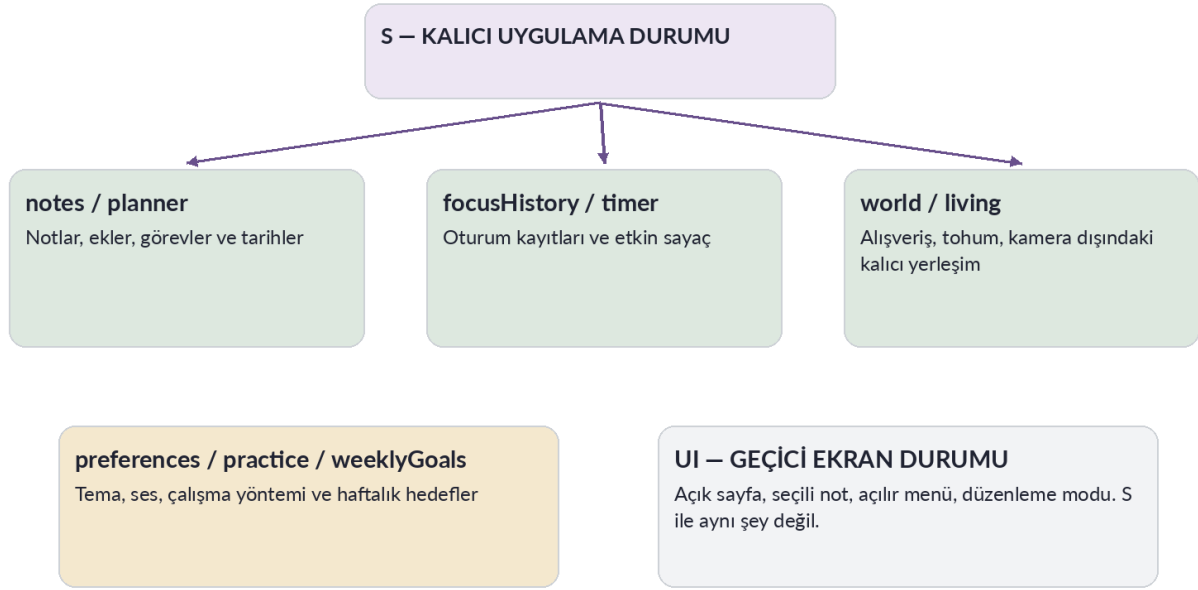
Bakım açısından sonucu

Bu yapı hızlı eklemeler yapmayı kolaylaştırır; fakat davranışı takip etmek için yükleme sırasını bilmek gerekir. İleride modüllere ayırma düşünülebilir. Bu rehber, kaynakta böyle bir yeniden yapılanma yapılmış gibi anlatmaz ve mevcut kodu yeniden düzenlemez.

Araştırma yöntemi: işlev adını bütün tools/ui içinde ara → build-ui.cjs listesindeki sıralamayı kontrol et → eski işlevin saklandığı değişkeni izle → en son atanan karşılığın hangi duruma göre eski işlevi çağırdığını oku.

Kaynak izi [K04, K12–K18] build-ui.cjs; stable/haven/atelier katmanları.

S ve UI: kayıt ile görünümün ayrılması



Şema 5. Kalıcı uygulama durumu ve geçici ekran durumu. Alanlar açıklama için gruplanmıştır.

S, uygulamanın saklanacak verilerinin merkezidir. Notlar S.notes içinde, takvim S.planner içinde, oturumlar S.focusHistory içinde durur. world çalışma kredileri, alışverişler, saksılar ve yerleşimle ilgilenir. living ise genişleyen bahçe, anılar, hava ve oda atamaları gibi ek alanlar içerir.

UI; açık sayfa, geçici seçim, editör odağı, filtre veya araç modu gibi görünüm ayrıntılarını taşır. Kamera büyüklüğü ve kaydırma da çizici nesnesinde tutulur. Her küçük parmak hareketinde bütün uygulama verisini diske yazmak gerekmez.

Kimlik ilişkisini gösteren öğretim örneği

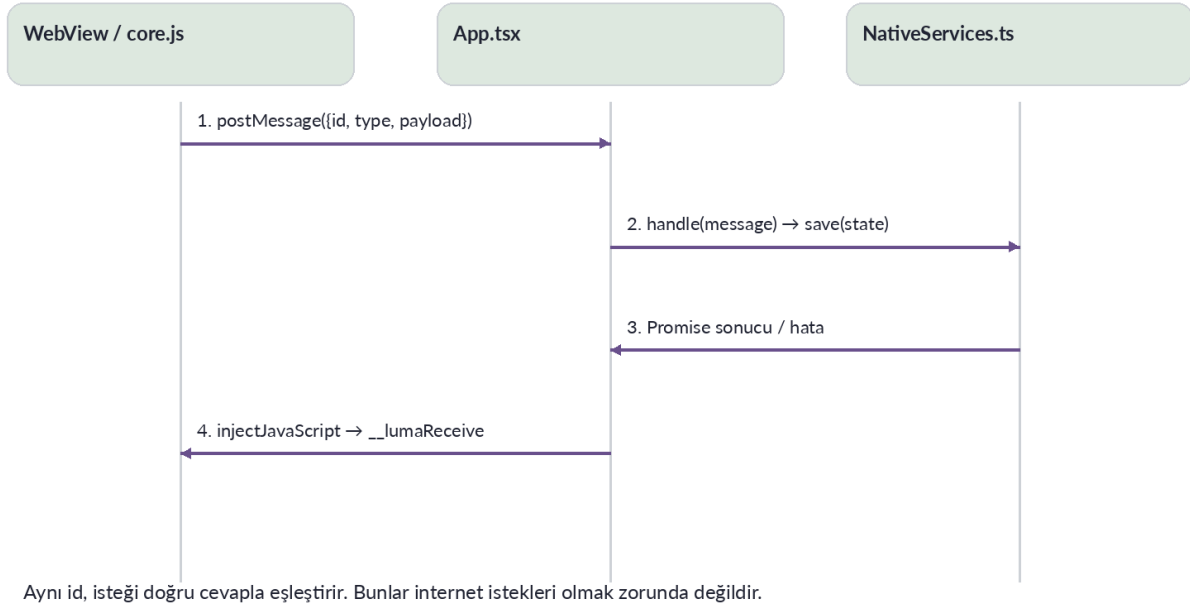
```
const durum = {
  notes: [{ id: "n1", title: "Bugün" }],
  planner: [{ id: "p1", noteId: "n1" }],
  focusHistory: []
};
```

Normalize etmek ne demek?

Eski veya dışarıdan gelen veri, son sürümün beklediği biçime uyarlanır. Eksik diziye varsayılan değer vermek, geçersiz ürün kimliğini ayıklamak, eski room:true kaydını salona eşlemek buna örnektir. Normalizasyon her açılışta yeni hediye üretmemeli veya bir eşyayı iki kez çoğaltmamalıdır.

Kaynak izi [K03, K05, K11, K13, K17] fresh/normalize; normalizeWorld; newLiving; rooms057.

Mesaj köprüsü: web görünümü telefonu nasıl çağırır?



Şema 6. Bir kayıt isteğinin kimliği ile yanıtın kimliğinin eşleştirilmesi.

WebView sayfası doğrudan telefon dosya yoluna yazmaz. Native.call, türü ve veriyi bir mesajın içine koyar. JSON metni `ReactNativeWebView.postMessage` üzerinden gönderilir. App.tsx mesajı alır; NativeServices uygun işi yapar. Sonuç, `replyTo` alanıyla aynı isteğe bağlanır.

Köprü biçiminin sadeleştirilmiş gösterimi; boş state gerçek kayıt için geçerli değildir

```
// Öğretim amaçlı mesaj biçimleri
{ "id": "istek-7", "type": "save", "payload": { "state": {} } }
{ "replyTo": "istek-7", "result": true }
// Hata yolunda result yerine error bulunabilir.
```

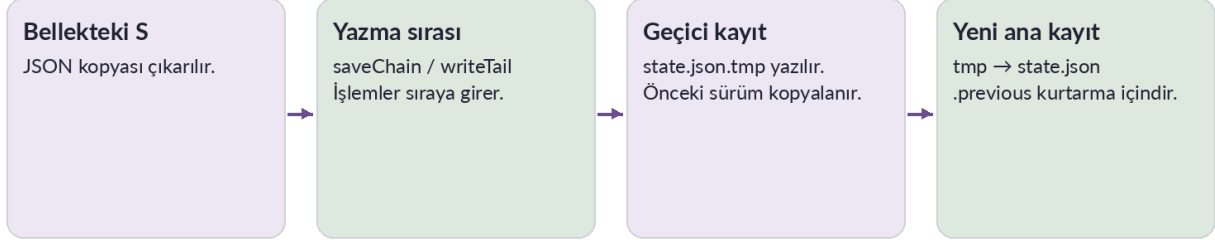
Birden fazla iş aynı anda bekleyebilir. id bu nedenle önemlidir: galeri sonucuyla kayıt sonucunu karıştırmamak gerekir. core.js bekleyen Promise'leri eşleştirir. App.tsx mesaj boyutunu ve bazı zorunlu alanları kontrol eder; servis de yalnızca tanıdığı işlem türlerini çalıştırır.

Telefon testi neden gerekli?

Tarayıcı önizlemesinde Native.call, browser.js yoluna gider. Görünüm aynı olsa bile işletim sisteminin izin penceresi, dosya seçici ve arka plana geçişi burada gerçek iPhone gibi çalışmaz.

Kaynak izi [K02, K03, K05, K23] `App.onMessage/send`; `Native.call/__lumaReceive`; `services.handle`. Genel köprü: [D02].

Not yazınca veri ne zaman diske gider?



Uygulama içi kurtarma dosyası, başka bir cihazda saklanan kullanıcı yedeği değildir.

Şema 7. Ekran olayından sıraya alınmış kayıt isteğine ve dosya değişimine kadar akış.

Kullanıcı yazarken her harfi ayrı bir dosya işlemi yapmak pahalı olabilir. core.js içindeki persist, normal çağrılarda yaklaşık 260 milisaniyelik erteleme kullanır. Buna debounce denir. Kullanıcı kısa aralıklarla yazmaya devam ederse önceki zamanlayıcı yenilenir; durunca son veri kaydedilir.

Kayıtlar ayrıca saveChain ile sıraya girer. NativeServices tarafında writeTail ve geçici dosya akışı vardır: veri önce .tmp dosyasına yazılır; önceki kayıt için .previous kopyası tutulur; yeni dosya asıl state.json yerine taşınır. Amaç yarım yazma durumuna karşı toparlanabilmektir. Bu tek başına bütün işletim sistemi kesintileri için mutlak garanti değildir.

Şu anki kayıt yolu

Kaynağın oluşturduğu yol • gerçek cihazda başlangıç dizini işletim sistemi tarafından belirlenir

```
FS.documentDirectory + 'lumanote-living-v05/state.json'
```

Dosya ekleri aynı kökün files/ dizinine kopyalanır. Eski lumanote-nocturne kayıtları için okuma/geçiş yolu bulunur. AsyncStorage paketi bulunmasına rağmen güncel ana not deposu bir AsyncStorage tablosu değildir; kaynakta asıl kayıt dosya sistemindeki state.json'dur.

Kaydedildi yazısı ne zaman doğru?

Yalnızca ekrandaki S nesnesi değişti diye işlem güvenli biçimde saklandı denmemelidir. Özellikle alışveriş ve ödülde kayıt hatasının geri alınması gerekir. “Yazılıyor”, “kaydedildi” ve “hata” durumlarını birbirinden ayırmak kullanıcının güvenini korur.

Kaynak izi [K03, K05] persist; NativeServices → load/save, ROOT/STORE/FILES.

Bir notun doğumu ve kaydı

Not eklerken önce benzersiz kimliği olan veri oluşturulur. Editör o notun HTML içeriğini düzenlenebilir alana koyar. Kullanıcı yazınca içerik yeniden okunur, izinli biçime temizlenir, düz metin özeti çıkarılır ve kayıt isteği sıraya alınır. Ekrandaki satır ile diskteki kayıt arasında birkaç farklı temsil bulunur.

Gerçek alan adlarıyla sadeleştirilmiş not nesnesi; tam şema değildir

```
const not = {
  id: "ornek-not-1",
  title: "JavaScript öğreniyorum",
  html: "<p>Bugün <strong>fonksiyonları</strong> çalıştım.</p>",
  plainText: "Bugün fonksiyonları çalıştım.",
  attachments: [],
  deletedAt: null
};
```

Neden HTML ve plainText birlikte var?

HTML kalınlık, renk, liste ve tablo gibi yapıyı taşır. plainText arama ve liste önizlemesi gibi yerlerde biçimsiz içerik sağlar. Birini değiştirip diğerini güncellemezsen notun içinde yeni yazı görünürken listede eski özet kalabilir. Kod değişikliği yaparken bu türetilmiş alan ilişkisini koru.

Seçim neden kaybolabiliyordu?

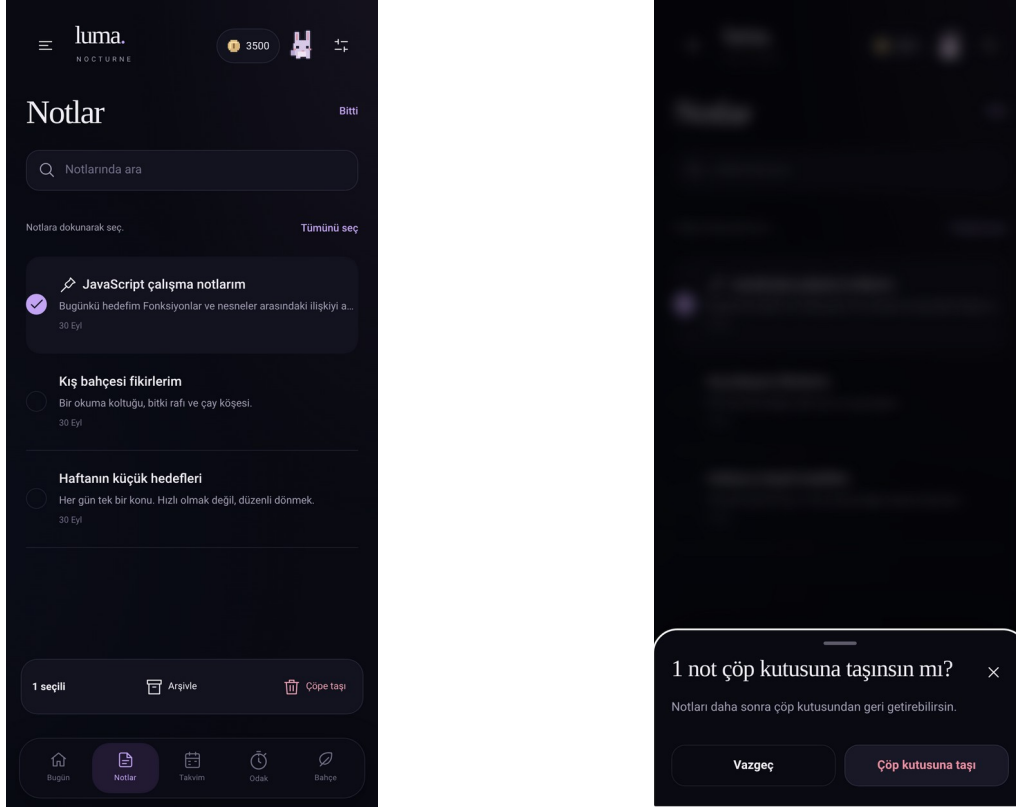
Metin seçimi DOM Range ile temsil edilir. Araç düğmesine dokunmak odağı başka öğeye taşıyabilir. polish-notes.js seçimi yakalar ve biçimlendirmeden önce geri getirir; bazı araçların pointer olayında odak kaymasını önler. Kaynaktaki biçimlendirme yolu document.execCommand kullanır. Bu rehber kaynakta başka bir editör kütüphanesi varmış gibi davranmaz.

Temizleme ile biçim desteği birlikte düşünülür

Bir rengin veya alt çizginin kayıttan sonra kaybolması yalnızca düğme hatası olmayabilir: sanitizeHTML ilgili etiketi/stili atıyor olabilir. İşlevi genişletirken güvenlik amacıyla konmuş temizleyiciyi tümünden kapatmak yerine izinli biçimleri incele.

Kaynak izi [K07, K08] editor.js; editor-v04.js; polish-notes.js → seçim/biçimlendirme; core.js → sanitizeHTML/plain.

Toplu seçim ve onay penceresi



Şekil 18. Seçim açıkken işlem çubuğu görünür; onay açılınca onun önüne geçmez.

Seçim modunda kimlikler bir Set içinde tutulur. Kullanıcı başka nota dokunduğunda kimlik eklenir veya çıkarılır. “Tümünü seç” görünür not listesinin kimliklerini alır. İşlem sırasında başlıklara göre değil kimliğe göre eşleştirme yapılması, aynı isimli iki notun yanlışlıkla karışmasını önler.

Gönderdiğin hata örneğinde toplu işlem çubuğu onay penceresinin üzerinde kalıyordu. haven-shell.js, menü açıkken arka içeriği etkileşimsiz yapar ve çubuğu/gezinmeyi gizler. Yalnızca CSS z-index değerini büyütme her zaman yeterli değildir: gözle görünmeyen bir öğe hâlâ tıklama yakalayabilir.

Silme işleminin üç seviyesi

Çöpe taşı: `deletedAt` işaretlenir. Geri getir: bu işaret kaldırılır. Kalıcı sil: kayıt gerçekten listeden çıkarılır. Arşiv ise bunlardan ayrı bir durumdur. Onay vermeden önce aday kimlik listesini almak, işlem sırasında görünüm değişse bile neyin seçildiğini açık tutar.

Kaynak izi [K08, K18] polish-notes.js → selection/bulk; haven-shell.js → sheet/closeSheet; tests/057.

El yazısı ve PDF: çizim katmanı



ÖNEMLİ AYRIM

Orijinal ek korunur. Dışa aktarılan kopya raster görüntülerden oluştuğu için aranabilir metin ve bağlantılar korunmaz.

Şema 8. Orijinal belge, işaretleme verisi ve paylaşım kopyası farklı nesnelerdir.

El yazısı ekranında bir çizgi, renk ve kalınlıkla birlikte nokta dizisi olarak saklanır. Noktaları 0-1 aralığına oranlamak, ekran genişlediğinde aynı çizginin uygun yere ölçeklenmesini sağlar. Ekranın sol üstünden 100 piksel yerine “genişliğin %25’i” demek buna örnektir.

Çizgi modelini anlatan sadeleştirilmiş örnek

```
const çizgi = {
  color: "#8961b8",
  width: 3,
  points: [{ x: 0.25, y: 0.4 }, { x: 0.30, y: 0.43 }]
};
```

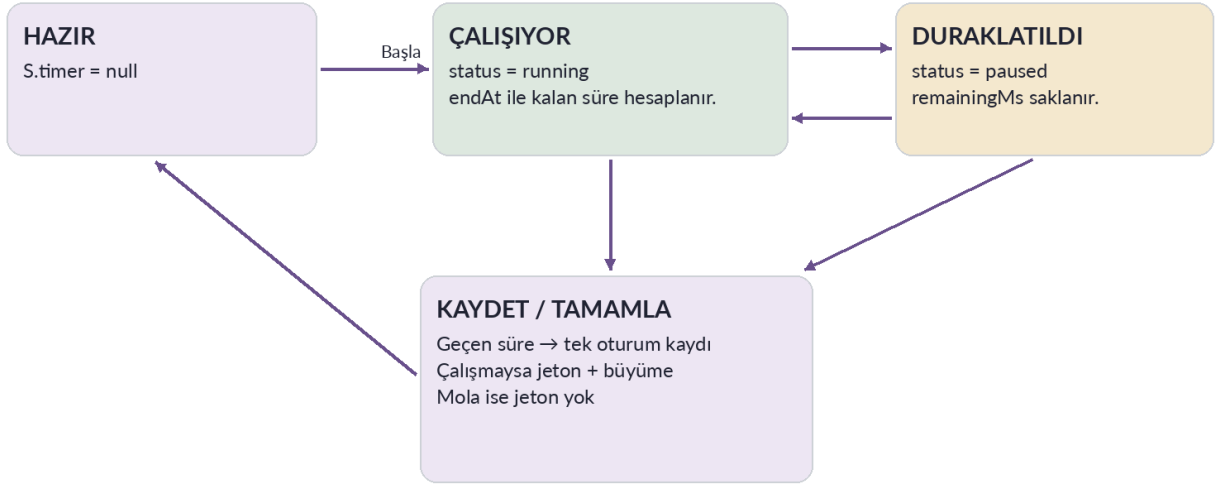
PDF tarafında pdfjs-dist sayfayı Canvas’a çizer; üstteki işaretleme katmanı kullanıcının çizgilerini tutar. İşaretleme dışı aktarmada pdf-lib ile yeni PDF üretilir. Bu sürüm sayfa görüntülerini kullanır: orijinal dosya değişmez ama çıkan kopyada arama yapılabilir metin ve bağlantılar korunmayabilir.

Kaynağın sınırları

Kod dosya ve sayfa sınırları uygular: PDF dosyası için 25 MB ve görüntülemeye 200 sayfa sınırı bulunur. Şifreli PDF için ayrı bir açma yolu istenir. Bu kitap gerçek iPhone’da PDF işaretleme ve dışa aktarma kabul testinin yerine geçmez.

Kaynak izi [K09] living-notes.js → ink; living-pdf.js → openPdf/renderPdfPage/exportAnnotatedPdf; [K04] PDF paketleme.

Sayaç: her saniye bir eksiltmek neden yetmez?



Animasyon yalnızca sonucu gösterir; süre ve bakiye hesabının kaynağı değildir.

Şema 9. Çalışma ve mola ayrı fazlardır. Duraklatma bir bitirme veya ödül anı değildir.

Bir sayaç yalnızca setInterval içinde sayıyı bir azaltırsa arka planda geciken çağrılar süreyi bozabilir. LumaNote kalan zamanı kayıtlı bitiş anıyla karşılaştırır. Aralık çağrısı ekranı yeniler; gerçek zamanın kaynağı çağrı sayısı değil saat farkıdır.

Geri sayım ilkesini gösteren öğretim örneği

```

const bitisAni = Date.now() + 25 * 60 * 1000;
function kalanMs() {
  return Math.max(0, bitisAni - Date.now());
}
  
```

Gerçekte endAt, remainingMs, planlanan süre, faz, durum ve kontrol noktaları birlikte kullanılır. Duraklatınca kalan değer tutulur; devam edince yeni bitiş anı hesaplanır. Flowtime ileri sayan görünüm verir; kaynakta bunun için de üst sınır ve oturum alanları bulunur. Bu, sınırsız çalışmayı teşvik eden veya doğrulayan bir ölçüm değildir.

İşlem sırası

stable-focus.js önce tek işlem kilidi alır, gerekli yeni durumu kurar ve kaydeder. Kayıt başarısızsa önceki duruma döner. Bildirim veya ses gibi yan işler kullanıcı düğmesini süresiz bekletmemelidir. Aynı session id geçmişte zaten varsa ikinci sonuç üretilmemesi ayrıca kontrol edilir.

Kaynak izi [K10] focus.js → remainingMs; living-focus.js; stable-focus.js → startFocus/endFocus/togglePause.

Jeton hesabı ve geliştirme bütçesi

LumaNote'un jetonları gerçek para değildir. Normal kazanç, kayıtlı çalışma saniyelerinden türetilir; mola süresi sayılmaz. world.credits, oturum kimliklerini saniyelerle eşler. Aynı oturumun birkaç kez normalize edilmesi yeniden kredi eklememelidir.

living-model.js hesabının Türkçe değişkenlerle açıklaması; test cüzdanı hariç

```
yeniSure = Math.max(0, toplamSaniye - eskiSurumBaslangici);
kazanilan = eskiBakiyeTabani + Math.floor(yeniSure / 20);
kalan = Math.max(0, kazanilan - alisverisToplami);
```

20 saniyede 1 jeton, dakikada 3 jeton demektir. Ancak eski sürümden taşınan bakiye tabanı korunur; eski bütün süreyi yeniden üçle çarpıp kullanıcıya ikinci kez ödeme yapılmaz. Bu nedenle mevcut kullanıcının toplam bakiyesi her durumda "bütün geçmiş dakika × 3" değildir.

Satın alma ve animasyon

Önce ürünün varlığı, fiyat, sahiplik ve bakiye kontrol edilir. Alışveriş kaydı oluşturulup kaydedilir. Ödül animasyonu da çalışma kaydı başarıyla tamamlandıktan sonra oynar. Animasyonu tekrar oynatmak yeni gelir kaydı oluşturmamalıdır. Ekrandaki sayı, verinin görsel sonucudur.

100.000.000 test jetonu

test-coins.js ayrı geliştirme bütçesini yönetir. Paketteki açma anahtarı kapalıdır; etkinlik için native geliştirme izni ve anahtar birlikte gerekir. Test parası önce harcanır; tekrar açılış yeni bütçe yaratmaz. Kapatma akışı test alışverişlerini ve ilişkili yerleşimleri temizlemek için ayrı izler tutar.

Yayın güvenliği açısından ayırım

__DEV__ kontrolü geliştirme davranışını ayırır; bu bir çevrimiçi hile önleme sunucusu değildir. Yerel kayıt dosyası düzenlenebilir bir veridir. Gerçek para, rekabet veya çok cihazlı ekonomi eklemek bu projede bulunmayan ayrı bir tasarım gerektirir.

Kaynak izi [K10, K11, K13, K29] world credits; living wallet; stable-focus coinCelebration; test-coins.js.

Tohumdan anı bahçesine veri akışı

Bir saksıya tohum ekerken duvar saatindeki tarih tek başına büyüme kaynağı değildir. Kaynak, o ana kadarki toplam çalışma saniyesini `atSeconds` olarak saklar. Daha sonra toplam çalışma arttıkça ekimden beri kazanılan süre bulunur.

`world.js` → `plantProgress` formülünün öğretici gösterimi

```
ilerleme = clamp(
  (toplamCalismaSaniyesi - ekimAnindakiSaniye)
  / (tohumDakikasi * 60),
  0, 1
);
```

`clamp` sonucu 0 ile 1 arasında tutar. 0 yeni ekim, 1 hazır bitkidir. Örneğin 25 dakikalık ayçiçeği, ekildikten sonraki 10 dakika kayıtlı çalışmada $10/25 = 0,40$ ilerler. Ekimden önceki saatler o yeni bitkiyi anında büyütmez. Üç saksı aynı toplam çalışma artışını kullanabilir.

Veri	Görevi
<code>world.plots</code>	Üç aktif yetiştirme yuvası; boş yuvada null.
<code>world.harvested</code>	Hazır olup bahçeye taşınan bitkiler.
<code>living.pendingWater</code>	Görünür alanda oynatılmayı bekleyen gelişim/sulama olayları.
<code>living.memorials</code>	Kullanıcının isim verdiği ayrı anı bitkileri.
<code>living.placement</code>	Sahnedeki bitki ve diğer nesnelerin konumları.

Sulama animasyonu gelişimin geri bildirimidir; tekrar tekrar dokunarak ücretsiz sınırsız süre kazanma yolu değildir. Hazır bitkiyi bahçeye taşımak saksıyı boşaltır. Anı bitkisi ise ayrı bir isimli kayıttır; onu taşımanın odak geçmişini silmesi gerekmez.

Bir yaşam döngüsünü test et

Boş saksı → ekim → kısmi gelişim → hazır → bahçeye taşı → yeniden aç. Her adımda hem veri alanına hem ekrana bak. “Çiçek göründü” tek başına saksının doğru boşaldığını ispatlamaz.

Kaynak izi [K11–K13, K16] `plantProgress/plantSeed/harvestPlant`; `queueGrowth`; `memorySection056`.

Piksel bahçe nasıl çiziliyor?

Bahçe bir video değildir. Kaynakta Canvas üzerinde çizilen zemin, yapılar, bitkiler, hayvanlar ve hava katmanları bulunur. Bazı model tarifleri SVG olarak da kullanılabilir. Mağaza küçük model görselini gösterirken bahçe aynı ürün kimliğinin sahne çizimini kullanır.

Basit bir piksel ağaç çizme örneği; kaynak modelinin kopyası değildir

```
const canvas = document.querySelector("canvas");
const ctx = canvas.getContext("2d");
ctx.imageSmoothingEnabled = false;
ctx.fillStyle = "#805337";
ctx.fillRect(44, 58, 12, 28); // gövde
ctx.fillStyle = "#749a60";
ctx.fillRect(26, 20, 48, 42); // yaprak kümesi
```

Canvas bir çizim yüzeyidir; fillRect renkli dikdörtgen çizer. Tarayıcı çizgi, şekil ve görüntüleri bu yüzeye işler. Canvas içindeki her ağaç otomatik olarak ayrı HTML düğmesi olmaz. Dokunma için ayrıca “bu noktada hangi nesne var?” sorusunu çözen hit test gerekir. [D05]

Çizim sırası derinlik hissi verir

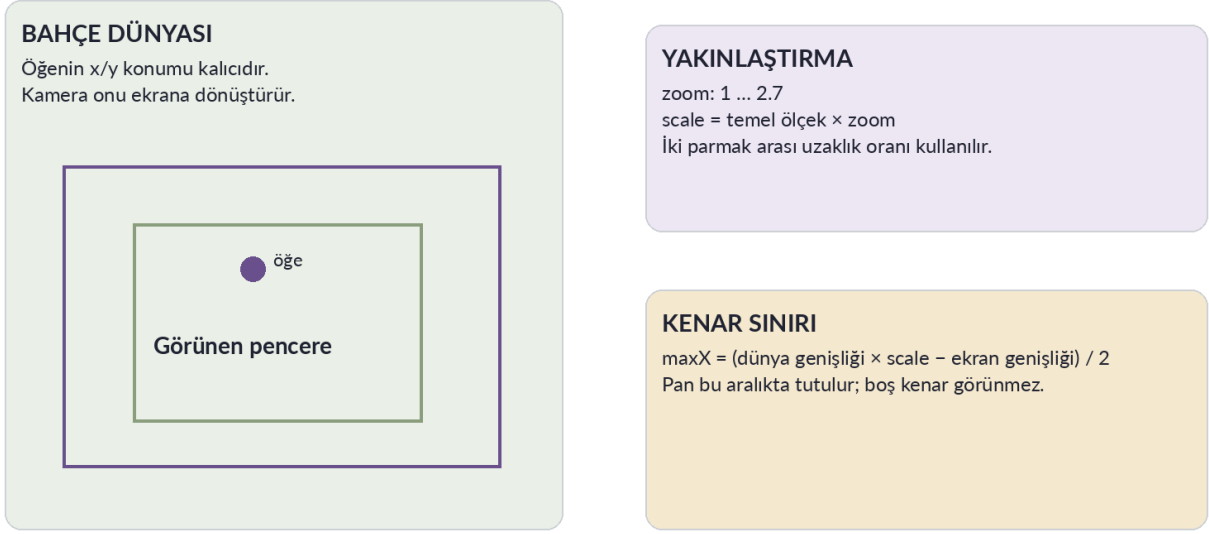
Zemin önce, eşyalar sonra, bazı hava efektleri en son çizilir. Nesnelerin ayak noktasına göre sıraya alınması, bir hayvanın ağacın önünden veya arkasından geçiyormuş gibi görünmesini sağlar. Renk seçimi, gölge ve birkaç piksel ışık aynı modelin düz görünmesini engeller; bunlar model tarifinin parçasıdır.

Hareket yaşam döngüsü

Sahne görünür değilken animasyonun durması pil ve işlem yükü açısından önemlidir. Stable katmanı görünürlük ve zaman aralığı kontrolleri ekler. Her animasyonu mutlaka 60 kare/saniye çizen bir oyun motoru varsayma; kaynak kendi zamanlamasını kullanır.

Kaynak izi [K12, K14–K17] sprites.js; living-garden.js ve sınıf katmanları; ...-art.js. Canvas kavramı: [D05].

Yakınlaştırma ve çerçeve sınırı



Şema 10. Sahne koordinatı, ölçek ve izinli kamera hareketi. Matematik örneği öğretim amaçlıdır.

Telefon ekranı ile bahçenin kendi koordinatları aynı değildir. Son dış bahçe 360×480 sahne alanını kullanır. Telefonun genişliği başka olabilir. Çizici, sahneyi çerçeveyi dolduracak ölçeğe çevirir; yakınlaştırma bu ölçeğin üzerine uygulanır.

Kameranın yatay sınırını anlatan sadeleştirilmiş formül

```
ölcek = Math.max(ekranW / dünyaW, ekranH / dünyaH) * zoom;
sinirX = Math.max(0, (dünyaW * ölcek - ekranW) / 2);
panX = clamp(panX, -sinirX, sinirX);
```

Yatayda hesaplanan kural dikeyde de uygulanır. Böylece zoom büyüyünce hareket edilecek alan açılır ama sahnenin dışındaki boşluk gösterilmez. Dokunulan ekran noktasından nesneyi bulmak için aynı dönüşüm ters yönde uygulanır: kaydırma çıkarılır, ölçeğe bölünür.

Eşya taşımak ve kamera taşımak

Kamera kayınca living.placement değişmemelidir. Düzenleme modunda bir eşyayı taşıyınca ise nesnenin kayıtlı koordinatı değişmelidir. Aynı parmak hareketi farklı modlarda farklı iş yapacağı için gesture ayrımı ve hit test, görsel çizim kadar önemlidir.

Kaynak izi [K12, K16, K17] stable-garden.js; atelier-garden.js; haven-garden.js; interiorPoint057.

Yeni ürün: isim eklemek neden yeterli değil?

Bir ürünün kullanıcıya çalışır biçimde ulaşması birkaç parçanın birlikte tamamlanmasını gerektirir. Sadece katalog adını yazarsan mağazada metin görünebilir ama yerleştirmede model çıkmayabilir. Sadece modeli çizersen ürün satın alınabilecek bir kimliğe sahip olmayabilir.

Parça	Soru	Kaynak örneği
Katalog kaydı	Kimlik, ad, fiyat, kategori nedir?	haven-model.js → FURNITURE057
Model tarifi	Mağazada ve sahnede nasıl çizilecek?	haven-art.js
Yer kuralı	Bahçe mi, oda mı; önerilen oda hangisi?	group, roomOnly, recommendedRoom
Satın alma	Zaten var mı, bakiye yeterli mi?	buyItem ve sonraki katmanları
Konum	Hangi alanda, hangi koordinatta?	world.layout; living.placement
Kayıt geçişi	Yeni kimlik yeniden açılışa geçerli mi?	normalizeWorld / newLiving
Test	Satın al, taşı, kaydet, yeniden açınca doğru mu?	tests/057

Gerçek katalog satırı • haven-model.js → FURNITURE057

```
ghChair057: ["Hasır okuma koltuğu", 550, 74, "greenhouse"]
```

Buradaki sıra ad, fiyat, çizim boyutu ve önerilen odadır. Dosyanın devamındaki döngü bunu DECOR içine yapılandırılmış nesne olarak aktarır. Dizinin 550 değerini sahnenin x koordinatı sanmak yanlış olur; aynı sayısal türün farklı bağlamları vardır.

186 sayısını nasıl okumalı?

Son çalıştırılmış katalogda DECOR toplam 186 anahtar içerir. Bu, 186 bağımsız bahçe mobilyası demek değildir: ev eşyaları, saksı seçenekleri ve özel hava satın alımları da aynı katalog içinde yer alır. Hayvanlar PETS, tohumlar SEEDS içindedir.

Kaynak izi [K11, K15–K17] DECOR/PETS/SEEDS; koleksiyon kayıtları; art ve satın alma işlevleri.

Odalar: nesne tek, bulunduğu yer değişken

Ev büyüdüğünde her oda için ayrı alışveriş listesi oluşturulmadı. Eşyanın sahipliği ortak kalır; bulunduğu oda bir kimlik eşleştirmesiyle belirtilir. Bu, koltuğu seradan salona taşıırken ikinci kez satın almamak ve iki odada birden çoğaltmamak için önemlidir.

Gerçek alan adlarıyla örnek oda kaydı; bütün alanlar gösterilmez

```
living.rooms057 = {
  version: 1,
  assignments: { "decor:ghChair057": "greenhouse" },
  styles: {
    greenhouse: { wall: "sage", floor: "stone" },
    salon: { wall: "cream", floor: "oak" }
  }
};
```

Geriyeye uyum

Önceki sürümde yalnızca room:true olarak işaretlenmiş eşya olabilir. newLiving yeni rooms057 yapısını tamamlar; tanımlı oda ataması yoksa eski ev eşyası salon olarak yorumlanır. Yeni sürümün açılması bütün eşyaları “kayıt yok” diye silmemelidir.

Aynı kayıt, farklı çizim dönüşümü

Eski kayıt koordinatları korunurken interiorPoint057 ve interiorLayout057 görünüm dönüşümünü yapar. Bir taraf eski yerleşimden yeni dikey odaya, diğer taraf sürüklenen görüntü noktasından kayıt uzayına gider. Dönüşümlerin birbirini terslemesi taşımanın tutarlı kalmasını sağlar.

Mini okuma görevi

haven-model.js içindeki itemSpace057 işlevini bul. room işareti yanı sıra eşyanın neden garden olarak yorumlandığını açıkla. Ardından openSpace057 işlevinde oda değişince neden seçili öge ve zemin fırçasının temizlendiğini düşün.

Kaynak izi [K17] SPACES057; itemSpace057; openSpace057; interiorPoint057/interiorLayout057.

Tema, atmosfer ve çizicinin yaşamı

Tema, arayüz renklerini ve efekt türünü seçer. Bahçe atmosferi ise dünyadaki mevsim/süs görünümünü etkiler. Manzara, zemin ve uzak plan paletini belirler. Bunlar tek bir “arka plan resmi” alanının farklı adları değildir; koddan birbirinden ayrı seçimlerdir.

Seçim	Örnek	Ana etkisi
Uygulama teması	Nocturne, Opal Akışı	Arayüz rengi, lava veya başka efekt.
Manzara	Ay Bahçesi, Güneşli Çayırı	Bahçe paleti, uzak plan ve sahne görünümü.
Atmosfer	Sakura, Yılbaşı	Ağaç, ev süsleri ve özel parçacıklar.
Hava / Işık	Kar / Gece	Yağış ve aydınlatma katmanları.

Lava tekniği

lava.js ekran kartında çalıştırılan GLSL shader kodu içerir. Bu kod her görüntü noktasının rengini hareketli şekillere göre hesaplar. Canvas tabanlı yedek yol, WebGL kullanılamayan durumlar için bulunur. Yeni Atölye temaları ek renk/parlama davranışları ekler; aynı shader’ın bütün cihazlarda aynı hızda çalışacağı varsayılmaz.

Bir eski hata ne öğretir?

Lava çizicisi, lava rengi olmayan bahçe temasına geçince eski veri alanını okumaya devam edebiliyordu. Çözüm yalnızca eksik alana rastgele renk yazmak değildi: eski çizici temizlenip uygun yeni çizici kurulmalıydı. Sahne sahibi değiştiğinde olay ve animasyon döngülerinin de sahibi değişmelidir.

Değişiklik deneme listesi

Yeni temayı aç → başka efekte geç → bahçe arka planıyla odak aç → tam ekrana gir/çık → hareketi kapat → tekrar aç. Tema tek bir güzel ekran görüntüsü değil, bu geçişlerin tamamında doğru çalışan bir sistemdir.

Kaynak izi [K14, K16, K18, K25] lava.js; scenes.js; atelier-themes.js; stable-shell.js; 0.5.3 raporu.

Ses: dosya üretimi ile oynatma farklı işler



BİR SESİN ÜÇ AYRI SORUSU

Dosya açılıyor mu? Teknik kontrol. Tını doğal mı? Dinleme testi. Kayıt mı sentez mi? Kaynak türü ve lisans metaverisi.

Şema 11. Üretilen MP3, build sırasında seçilir; telefon ve tarayıcı farklı oynatma yolları kullanır.

Python üretim betikleri nota olaylarından veya sayısal gürültü/zarflardan ses örnekleri oluşturur. FFmpeg bu örnekleri MP3'e dönüştürür. Telefon her açılışta bu besteleri baştan üretmez; pakette hazır bulunan MP3 dosyasını oynatır. "Üretici kodu var" ve "varlık dosyası paketlenmiş" iki ayrı denetimdir.

AudioEngine.tsx yalnızca seviyesi açık kanallar için Track bileşenleri oluşturur. Her kanalda iki oynatıcı kullanarak dosya sonundan başına geçişi yumuşatır. Birinin sesi azalırken diğerinin artması crossfade olarak adlandırılır. Kaynakta yaklaşık 1,5 saniyelik geçiş ve genel seviye dengelemesi bulunur.

AudioEngine.tsx içindeki ilgili gerçek ses ayarları

```
shouldPlayInBackground: false,
playsInSilentMode: true,
allowsRecording: false
```

Bu ayarlar mikrofonla kayıt yaptığın anlamına gelmez. App.tsx ayrıca arka plana geçişte playing durumunu kapatır. Dolayısıyla "sessiz modda çalabilir" ile "telefon kilitliken devam eder" aynı özellik değildir.

Kaliteyi doğru raporla

Ses dosyasının çözülebilmesi ve kırılmamış örneklerden oluşması teknik kontroldür. Doğal, hoş veya yeterince farklı bulunması dinleme değerlendirmesidir. Kaynaktaki rapor öznel fiziksel dinleme yapıldığını söylemez; bu rehber de bunu varsaymaz.

Kaynak izi [K02, K22, K27, K28] AudioEngine.tsx; AudioAssets.ts; generate-piano-057.py; ses manifestleri.

Takvim tekrarı ve yerel hatırlatma

Bir planın başlığı, tarihi, saati, süresi ve tekrar kuralı vardır. Haftalık tekrarlanan bir plan için kullanıcıya her hafta tamamen bağımsız yeni plan yazdırılmaz; belirli tarih aralığında oluşan örnekler hesaplanır. Tamamlanma bilgisi de tekrarın ilgili günüyle ilişkilendirilebilir.

Plan tipindeki gerçek alanlarla öğretim örneği

```
const plan = {
  id: "p1", title: "Okuma", kind: "study",
  date: "2026-10-01", time: "18:00",
  durationMinutes: 25,
  repeat: "weekdays", reminderMinutes: 10,
  completedDates: {}, noteId: "n1"
};
```

planner.js tekrar örneklerini üretir. NativeServices gelecekteki uygun hatırlatmaları sınırlı sayıda planlayıp mevcut bildirim kimlikleriyle uzlaştırır. Kaynakta en yakın 48 anımsatıcıya ayrılmış sınır ve oturum/mola işleri için pay vardır. Telefonun uygulama izni kapalıysa takvim kaydı var olsa bile bildirim beklediğin gibi teslim edilmeyebilir.

Uzak push ile aynı şey değil

Buradaki planlanan hatırlatmalar cihazın kendi yerel bildirimleridir. Bu paket ayrı bir sunucudan kullanıcılara toplu push gönderen altyapı içermez. Expo Go uyarısında uzak Android push kısıtının görünmesi ile kullanıcının bütün yerel plan verisinin bozulması aynı sonuç değildir.

Takvime dışa aktarma

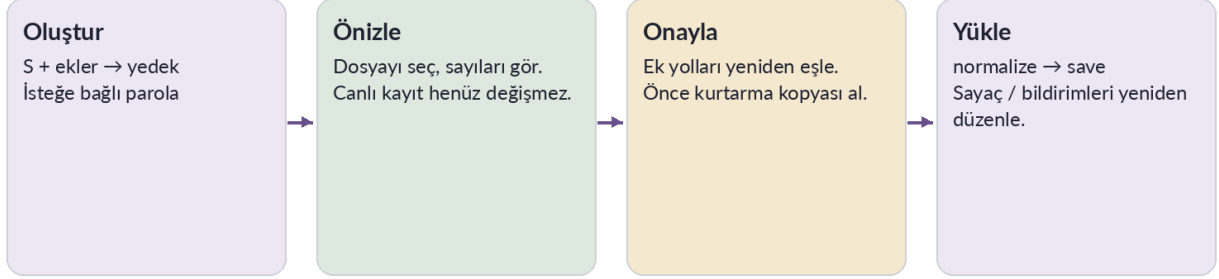
ICS dışa aktarma işlevi vardır. Bu bir takvim dosyası üretme yoludur; Google Calendar veya Apple Calendar ile çift yönlü sürekli senkronizasyon değildir. Not ile plan arasındaki noteld ilişkisinde de notun sonradan çöpe taşınması gibi durumlar ayrıca ele alınmalıdır.

Tarih testi için üç senaryo

Hafta içi tekrarın cumartesi görünmemesi; eski başlangıç tarihinden önce örnek üretilmemesi; plan değişince eski bildirim kimliklerinin başıboş kalmaması. Saat farkı ve gerçek cihaz izinlerini ayrıca sınamak gerekir.

Kaynak izi [K05, K19] planner.js; NativeServices → occurrences/refreshReminders/reconcile; Plan tipi.

Yedek: içerik paketi ve şifreli zarf



Şifreli dışa aktarım, uygulamanın canlı state.json dosyasını şifrelemez.

Kodu ZIP'lemek ile kişisel veriyi dışa aktarmak iki ayrı yedekleme işidir.

Şema 12. Veri yedeği oluşturma ve onaylı geri yükleme. Kod yedeği ayrı bir işlemdir.

Yedek paketi yalnızca state.json kopyası değildir. Ulaşılabilen dosya ekleri Base64 olarak eklenir; erişilemeyenlerin adları ayrıca listelenir. Kaynak, gömülü dosyalar için toplam 80 MB sınırı uygular. Base64, ikili dosyayı metne çevirir; şifreleme değildir.

Parolalı seçenekte BackupCrypto.ts, PBKDF2-SHA256 ile paroladan anahtar türetir; 600.000 yineleme, 16 bayt rastgele salt ve 32 bayt anahtar kullanır. AES-256-GCM ile paket şifrelenir. 12 bayt nonce, ek doğrulanan veri ve sürümlü zarf biçimi vardır. Bunlar dosyada görülen parametrelerdir; bağımsız bir güvenlik denetimi sonucu değildir.

En önemli iki sınır

Şifreleme dışa aktarılan dosyaya uygulanır; canlı yerel kayıt dosyasına değil. Parola unutulduğunda onu sunucudan sıfırlayan LumaNote hesabı yoktur. Sırf bu özelliği görmek için tek canlı veri kopyan üzerinde geri yükleme denemesi yapma.

İçe aktarma önce dosyayı okur, doğrular ve önizleme için bekletir. Onaylanan dosyaların yolları uygulamanın kendi files/ alanına taşınır; URI eşleştirmeleri güncellenir. Son aday veri normalize edilip kaydedilir. Bu süreç iki yedeği akıllıca birleştirmez; mevcut verinin yerine geçer.

Kaynak izi [K05, K06, K21] createBackup/previewBackup/commitBackup; BackupCrypto.ts; backup.js.

Projeyi çalıştırma mantığı

Mevcut çalışan uygulamayı kullanmak, yeni bir kaynak klasörünü ilk kez kurmak ve bir sürüm güncellemesi uygulamak farklı işlemlerdir. Klasör adlarının benzer olması onları aynı proje yapmaz. Terminal'de gördüğün yol, hangi dosyaların paketlenildiğini anlamak için ilk kontroldür.

Salt kontrol komutları · dosya silmez veya paket kurmaz

```
pwd
node -v
npm -v
```

pwd bulunduğu klasörü gösterir. node -v ve npm -v kullanılan araç sürümlerini yazar. % işareti ve kullanıcı adın komutun parçası değildir. Bir kod bloğunu yapılandırırken başına ^[[200~ benzeri kontrol karakterleri gelirse o satır düzgün çalışmamış olabilir.

0.5.7 paketinin iki yolu

Amaç	Paketteki dosya	Dikkat
Ayrı tam projeyi çalıştır	BASLAT.command	İlk kurulum ağ/paket indirimi ister. Kendi klasöründe çalışır.
Mevcut projeyi güncelle	MEVCUT_PROJEYI_DUZELT.command	Varsayılan hedef Masaüstü/LumaNote_Yasayan_Bahce. Önce kod yedeği ve hash kontrolleri.
Yerel ağ yerine tünel	TUNEL_ILE_BASLAT.command	Ayrı bağlantı yöntemi; yeni QR kodu kullanılmalı.

Kaynak paketin ayrı ilk çalıştırma yolu · mevcut projenin yerine otomatik veri taşımaz

```
bash "$HOME/Desktop/LumaNote_057_Tam_Proje/BASLAT.command"
```

Başlatıcı Node 24 ortamını npm üzerinden kullanır. setup.cjs eksik kurulumda npm install, Expo paket uyarlamaları, arayüz üretimi ve doğrulama yapar. Sonra --go --clear --port 8098 ile sunucuyu açar. Terminal açık kalırken yeni QR okutulur. Hazır telefon kayıtlarının ayrı proje kimliğinde otomatik görünmesini varsayma.

Kaynak izi [K01, K24] BASLAT.command; TUNEL_ILE_BASLAT.command; setup.cjs; README_TR.md. Genel Expo akışı: [D01].

Bir değişiklik yapmanın güvenli döngüsü

Öğrenirken doğrudan günlük kullandığın kayıtları taşıyan tek klasörde deneme yapmak yerine kaynak kopyasında çalış. Bu rehber kaynak projeyi değiştirmedir. Yanındaki alıştırma paketi de gerçek LumaNote kayıtlarına erişmez.

Aşama	Ne yaparsın?	Ne kontrol edersin?
1. Küçük hedef	Örneğin bir kart başlığını değiştir.	Aynı anda beş farklı özellik değiştirme.
2. Doğru kaynak	tools/ui içindeki son etkili işlevi bul.	Üretilmiş UI_HTML.ts değil kaynak dosya.
3. Yeniden üret	npm run build:ui	Derleme çıktısı ve sürüm işareti.
4. Kaynak doğrula	npm run verify	Sözdizimi, varlık ve beklenen işaret kontrolleri.
5. Görünüm dene	Örnek veriyle sayfayı aç.	Mobil boyut, uzun yazı, boş durum.
6. Kayıt dene	Değişiklik yap; kapatıp tekrar aç.	Veri gerçekten korunuyor mu?
7. Cihazda dene	Expo Go / uygun test uygulaması.	Gerçek dokunma, klavye, izinler.

Proje scripts alanındaki gerçek komutlar • typecheck sonucu ayrıca değerlendirilmelidir

```
npm run build:ui
npm run verify
npm run typecheck
```

Bir kontrol komutunun tanımlı olması, bütün kontrollerin her ortamda önceden geçtiği anlamına gelmez. Özellikle typecheck, native API tipleri ve kurulu paketlere bağlıdır; bu belge bunun tüm proje için çalıştırılıp geçtiğini iddia etmez. Kaynak doğrulayıcısı da davranış testlerinin yerini tutmaz.

Tek değişiklik, tek gerekçe

Değişiklikten önce ekranı veya sonucu kaydet. Sonra yalnızca hedef davranışın farkını incele. Yeni bir düğme eklerken not kaydının, geri almanın veya kapatma hareketinin istemeden değişmediğini ayrıca kontrol et.

Kaynak izi [K01, K04, K24, K25] package.json scripts; verify.cjs; kaynak/test akışları.

Hata mesajını okumayı öğrenmek

Terminal'de WARN görmek her zaman uygulamanın durduğu anlamına gelmez. ERROR veya başarısız komut daha ciddi olabilir; ancak en sondaki hata bazen ilk sorunun sonucudur. Önce hatanın ilk oluştuğu satırı, dosya yolunu ve hangi işlemten sonra çıktığını kaydet.

Görülen durum	Nasıl yorumlamalı?	İlk güvenli kontrol
Eski ekran açılıyor	Başka proje sunucusu veya eski üretilmiş çıktı olabilir.	Terminal yolu, sürüm etiketi, build:ui çıktısı.
Cannot find module...	Gereken modül bu kurulumda bulunamamış.	İlk hata, package.json ve kurulu paket çözümlemesi.
transformFile undefined	Önceki transformer kurulum hatasının sonucu olabilir.	Üst satırlarda ilk eksik modülü ara.
Paket exports uyarısı	İçe aktarma yolu dışı aktarma sözleşmesiyle eşleşmiyor olabilir.	metro.config.js ve gerçek import yolu.
[LumaNote UI]...	WebView hatası native Terminal'e aktarılmış.	Ekran ve olay bilgisiyle ilk mesajı sakla.
Stopped server	Geliştirme sunucusu durmuştur.	Komut satırına dönüldü mü; yeniden başlatma gerekir mi?

Bu projede yaşanan öğretici örnek

Sohbetteki eski tanı çıktısında önce babel-preset-expo bulunamaması, ardından transformFile hatası vardı. Eksik ön uyarı görmek asıl nedeni bulmaya yardımcı olmuştu. Bu tarihsel olay “her transformFile hatasının nedeni aynıdır” demek değildir. Bugünkü hatayı eski teşhise bakmadan okumak gerekir.

Bir hata kaydı dört şeyi içersin

Sürüm etiketi + açtığın ekran + dokunduğun düğme/işlem + ilk hata satırları. Kişisel not metni veya gizli anahtarları hata raporuna gereksiz yere ekleme. Sorunu rastgele klasör silerek gidermeye başlamadan önce bu bilgileri topla.

Kaynak izi [K01, K02, K24] metro.config.js; App.tsx diagnostic; eski LumaNote-hata çıktısı bağlamsal örnektir.

Testler neyi kanıtlar, neyi kanıtlamaz?

Düzey	Örnek	Tek başına kanıtlamadığı
Sözdizimi / kaynak	JS ayrıştırma, model işaretleri, dosya hashleri.	Düğmenin kullanıcı açısından doğru çalışması.
İşlev / durum	Bir oturuma tek ödül, yetersiz bakiyeyi reddetme.	Gerçek iPhone dosya sistemi ve bildirim teslimi.
Tarayıcı etkileşimi	Tıklama, sürükleme, menü katmanı, ekran taşması.	WKWebView klavyesinin birebir davranışı.
Dosya / güncelleyici	ZIP yedeği, bilinmeyen kodda durma, geri alma.	Mac'teki her olası kurulum durumu.
Ses çözme	MP3 açılır mı; örnekler sonlu ve sessiz değil mi?	Müziğin kullanıcıya doğal ve hoş gelmesi.
Fiziksel kabul	Telefonda izin, yön, dokunma, kayıt, ses.	Bütün gelecekteki cihaz/sürüm koşulları.

0.5.7 arşivindeki rapor 190 doğrulama bildirir. Bunlar 67 arayüz/durum, 18 etkileşim, 25 güncelleyici, 18 ses ve 62 kaynak kontrolü olarak gruplanmıştır. Raporda tarayıcı localStorage için bellek adaptörü kullanıldığı ve fiziksel iPhone testlerinin yapılmadığı açıkça yazılıdır. Bu sayı bağımsız güvenlik sertifikası veya 190 ayrı telefon denemesi değildir.

Bu rehber hazırlanırken

Paketin kaynak doğrulayıcısı yeniden çalıştırıldı: 62 kontrol geçti. Gerçek HTML örnek kayıtlarla açılıp sayfa görselleri alındı; yakalanmamış JavaScript hata listesi boştu. Bu çalışma bütün 190 testi yeniden koşturma, gerçek Metro derleme veya telefon ses dinleme denemesi olarak sunulmaz.

İyi test bir beklenen sonuç söyler

“Mağazayı açtım” yerine “350 jetonla 550 jetonluk koltuğu alamamalıyım; bakiye ve satın alma listesi değişmemeli” de. Önce beklenen sonuç, sonra işlem, sonra veri ve görünüm karşılaştırması gelir.

Kaynak izi [K25, K26] docs/TEST_RAPORU_057.md; tests/057; bu belge için current-verify.txt ve yakalama kaydı.

Yedekli güncelleyiciyi anlamak

MEVCUT_PROJEYI_DUZELT.command bütün dosyaları körlemesine kopyalayan bir komut değildir. tools/repair-existing.cjs hedefi ve kaynakları kontrol eder. İzin verilen önceki dosya hashleriyle karşılaştırma yapar; bilinmeyen kişisel değişiklikte durur. Amaç senin değiştirdiğin kodun fark edilmeden silinmesini önlemektir.

Hash nedir?

Bir dosyanın içeriğinden hesaplanan kısa bir parmak izi gibi düşünebilirsin. Bir karakter değiştiğinde hash de değişebilir. Güncelleyici bunu “beklediğim eski dosya mı?” sorusu için kullanır. Hash dosyanın iyi kod olduğu veya güvenli olduğu anlamına gelmez; içerik eşitliğini denetleme aracıdır.

Aşama	Koruma amacı
Hedef klasörü doğrula	Yanlış projeye veya beklenmeyen yola yazmamak.
Kaynak / eski sürüm hashlerini denetle	Bozuk paketi veya kişisel değişikliği fark etmek.
Kod ZIP'ini oluştur ve doğrula	Geri dönülebilecek bir dosya kopyası almak.
Belirlenen dosyaları değiştir	Paket kilidi ve kimlik gibi alanları gereksiz yere yeniden kurmamak.
Son doğrulamayı çalıştır	Beklenen arayüz ve varlıkların oluştuğunu kontrol etmek.
Hata durumunda geri al	Kısmi değişmiş kaynak yerine önceki dosyaları korumak.

Kendi kodunu yazmaya başladığında

Güncelleyicinin senin değişikliğinde durması hata olmak zorunda değildir; bu tasarlanan korumadır. Uyarıyı aşmak için hash kontrolünü silmek yerine kaynak farkını karşılaştırmak gerekir. Projeyi geliştirecek kişi sürümleme ve değişiklik kaydı tutmalıdır.

Kod ZIP'i ile telefonun kişisel veri yedeği yine farklıdır. Kaynak yedeği not içeriklerini otomatik içermez. Test cüzdanının açık/kapalı anahtarını koruma da yeni bir 100 milyon bütçe verme işlemi değildir.

Kaynak izi [K24, K29] repair-existing.cjs; docs/repair-057.json; TEST_JETONLARI.command.

Yanındaki küçük alıştırma atölyesi

Bu rehberle eşlik eden LumaNote_Ogrenme_Atolyesi.zip, gerçek projeden bağımsız küçük örnekler içerir. Amaç yüzlerce dosya arasında kaybolmadan birkaç temel davranışı kendin değiştirebilmendir. Hiçbiri Expo uygulamasının kişisel kayıt alanına veya test jetonlarına erişmez.

Dosya	Öğrenilen konu	Çalıştırma
01-hizli-not.html	Form, DOM, dizi, seçim, yerel tarayıcı kaydı.	Tarayıcıda aç.
02-sayac.html	Bitiş zamanı, duraklatma, tek ödül.	Tarayıcıda aç.
03-mini-bahce.html	Canvas, ölçek, sınırlı pan, seçili nesne taşıma.	Tarayıcıda aç.
04-tipler.ts	Küçük not ve oturum türleri.	Metin olarak incele; Node doğrudan çalışma iddiası yok.
README.md	Güvenli denemeler ve beklenen sonuçlar.	Metin düzenleyicide aç.

HTML örneklerinde dosyaya çift tıklamak yeterli olabilir. Tarayıcın yerel dosyada saklamayı engellerse ilk örnek bunu bildirip bellek içi çalışır. Bu, LumaNote'un native dosya kaydını taklit ettiği anlamına gelmez. Her örneğin "Öğrenme ortamı" başlığı gerçek uygulamayla karışmasını önler.

Deneme yöntemi

Önce dosyayı değiştirmeden çalıştır. Ardından yalnızca bir değer veya bir CSS kuralı değiştir. Tarayıcıyı yenile ve farkı gözle. Sonucu tahmin etmeden onlarca satır kopyalamak yerine "bu değişiklik hangi veriyi etkileyecek?" sorusuna cevap ver.

Kapsam

Bu örnekler sıfırdan öğretim için yazılmış küçük uygulamalardır. Şifreli yedek, çoklu oda, gerçek PDF, işletim sistemi bildirimi veya hile önleme içermez. Buradaki bir fonksiyonu doğrudan büyük projedeki eş adlı fonksiyonun yerine koyma.

Kaynak izi [Ö01-Ö04] Bu rehberle oluşturulan bağımsız öğrenme örnekleri; kaynak projenin parçası değildir.

Atölye 1: ilk notunu kodla

01-hizli-not.html dosyasını aç. Bir başlık yazıp Ekle'ye bas. Yeni satırı seç ve seçilileri sil. Sayfayı yenileyerek tarayıcı depolamasının kullanıldığı ortamda kaydın kaldığını gözlemlle. Bu küçük örnek, gerçek not editörünün zengin metin kısmını değil temel liste-veri ilişkisini öğretir.

Atölyedeki işlemin sadeleştirilmiş özeti

```
const yeniNot = {  
  id: crypto.randomUUID(),  
  title: baslik.trim()  
};  
notlar.push(yeniNot);  
kaydet();  
ciz();
```

Görev A • Boş veri

Ekle'ye boş metinle bas. Yeni not oluşmamalı. Ardından yalnızca birkaç boşlukla dene. trim boşlukları temizlediği için yine reddedilmeli. Bu küçük kontrol, yüzlerce “Başlıksız” kayıt üretmemek için veri sınırının önemini gösterir.

Görev B • Kimlik

Aynı başlığa sahip iki not ekle. Birini seçip silince diğeri kalmalı. İşlem başlığa değil kimliğe göre yapılmalıdır. Kaynakta id alanının neden bulunduğunu gerçek bir davranışla öğrenmiş olursun.

Görev C • Görünüm

Kart rengini veya düğme köşe yarıçapını değiştir. Not verisi aynı kalırken görünüm değişmelidir. Böylece CSS düzenlemesi ile kayıt modeli değişikliğini ayırırsın.

Kabul ölçütü

Boş not reddediliyor; aynı başlıklı iki not ayrı kimlikle var; birini silince diğeri kalıyor; depolama hatası olduğunda kullanıcı sessizce yanıtılmıyor. Renk değiştirmek notları silmiyor.

Kaynak izi [Ö01] 01-hizli-not.html. Gerçek proje bağlantısı: [K03, K07, K08].

Atölye 2: sayaçtan sonuca

02-sayac.html denemeyi hızlandırmak için saniye cinsinden süre ister. Bu gerçek LumaNote'un ödül deposu değildir. Amaç tamamlama olayının tek kez oluşması ve duraklatmada kalan sürenin korunmasıdır.

Zamanı güncelleme ilkesinin sadeleştirilmiş özeti

```
if (durum !== "calisiyor") return;
kalan = Math.max(0, bitis - Date.now());
if (kalan === 0) bitirVeKaydet();
```

Görev A • 20 saniye

20 saniyelik oturum başlat. Sonunda 1 örnek jeton gösterilmeli. Düğmeye tekrar basmak aynı oturum için ikinci jeton üretmemeli. Ödül hızını artırarak denemek yerine kısa süreyi kullan; gerçek çalışma kayıtlarıyla karıştırma.

Görev B • Duraklat

Başlat, birkaç saniye sonra duraklat ve 5 saniye bekle. Kalan değer azalmamalı. Devam edince yeni bitiş zamanı kalan süre üzerinden kurulmalı. İlk başlangıç anını değişmeden kullanırsan beklenen sonuç bozulabilir.

Görev C • Kısmi kayıt

Daha uzun oturumda Bitir ve kaydet'e bas. Ödül planlanan tam süreden değil gerçekten geçen etkin süreden hesaplanmalı. Molayı taklit etmek için örneğe ayrı bir phase alanı eklemeyi düşün; bu alan break ise neden ödülün 0 olması gerektiğini yaz.

Bir üst adım

Uygulamayı kapatıp açma toparlaması bu küçük örneğin kapsamına dahil değildir. Gerçek LumaNote bunun için timer kaydı, endAt, remainingMs, kontrol noktaları ve persist zinciri kullanır. Ekranda çalışan mini sayaçla kalıcı mobil oturum aynı karmaşıklıkta değildir.

Kaynak izi [Ö02] 02-sayac.html. Gerçek proje bağlantısı: [K10, K11, K13].

Atölye 3: bahçenin sınırlarını çiz

03-mini-bahce.html basit bir Canvas sahnesi sunar. Yakınlaştırma kaydırıcısını büyüt, sahneyi sürükley ve kenarda durduğunu gör. “Nesneyi taşı” modunda ise kamerayı değil küçük bitkinin konumunu değiştirmeyi dene. Uygulamanın özgün piksel modelleri bu örneğe taşınmadı.

Sınırlandırma işlemi · x ve y için ayrı uygulanır

```
function kisitle(deger, alt, ust) {
  return Math.max(alt, Math.min(ust, deger));
}
const maxPan = Math.max(0, (sahneBoyut * olcek - alanBoyut) / 2);
pan = kisitle(pan, -maxPan, maxPan);
```

Görev A · Önce ölç

Yakınlaştırma 1 iken pan sınırı ne kadar? Görünüm sahneyle aynı orandaysa 0 olabilir. Yakınlaştırmayı 2 yaptığında fazla alan artar. Sınırsız sürükleme neden boş arka plan gösterir, kendi cümlemlerle açıkla.

Görev B · Ters dönüşüm

Fare veya parmak ekran koordinatı verir. Bitkiyi sahneye koymak için çerçevenin başlangıcını ve kamera kaymasını çıkarıp ölçüğe bölmek gerekir. Sadece ekran koordinatını kaydedersen yakınlaştırma değiştiğinde bitki atlayabilir.

Görev C · Seçim ile gezinme

Öğeyi taşıırken kamera sabit kalmalı. Gezinme modunda sürüklerken bitkinin dünya koordinatı değişmemeli. Aynı pointer olayı iki farklı niyete göre ayrılır. Kaynak projenin düzenleme modu da benzer bir ayırım gerektirir.

Kabul ölçütü

Dört kenarda boşluk yok; yakınlaştırma geri alınabiliyor; bitki dışarı sürüklenemiyor; parmak bırakıldığında hareket sona eriyor; pointercancel durumunda sürükleme takılı kalmıyor.

Kaynak izi [Ö03] 03-mini-bahce.html. Gerçek proje bağlantısı: [K12, K16, K17].

Gerçek projeye özellik ekleme planı

Temel örnekleri anladıktan sonra kaynak kopyasında küçük bir “Yeni çalışma kartı” veya “Yeni dekor” denemesi yapabilirsin. Burada hazır büyük bir kod bloğu yapıştırmak yerine özelliğin sözleşmesini yazmak daha öğreticidir.

Soru	Örnek: yeni okuma lambası
Kimliği ne?	Tek ve kalıcı id; görünen Türkçe addan bağımsız.
Verisi nerede?	DECOR kaydı, fiyat ve koleksiyon alanı.
Nasıl görünecek?	Mağaza küçük modeli ve sahne çizim tarifi.
Nerede kullanılacak?	Ev/sera kuralı; önerilen alan; boyut sınırı.
Nasıl alınacak?	Var olan buyltem akışı; yeni bağımsız cüzdan yok.
Nasıl saklanacak?	Mevcut satın alma ve konum kayıtlarıyla uyum.
Başarısız olursa?	Kayıt reddinde bakiye ve sahiplik geri dönmeli.
Hangi testler?	Yeterli/yetersiz bakiye, iki kez alma, taşıma, yeniden açma.

Yeni kaynak dosyası eklersen build-ui.cjs sırasını unutma. Kaydettiğin yeni veri alanı normalize edilirken atılmamalı. Var olan bir kimliğin anlamını tamamen değiştirmek de eski kullanıcının eşyasını başka ürüne dönüştürebilir. Gerekirse sürümlü veri geçişi tasarla.

Mevcut sürümün koruması

Kaynak değişikliği yaptıktan sonra eski onarıcı “kişisel değişiklik var” diye durabilir. Bu yüzden eğitim denemesini çalışma kopyasında tut. Onarıcı manifestini rastgele güncellemek veya korumayı kapatmak öğrenmenin ilk adımı değildir.

Tamamlandığında yalnızca yeni ürünün görüldüğü ekranı değil, satırların ne yaptığını açıklayan kısa bir değişiklik notu yaz. Bir başkasına neden o dosyalara dokunduğunu anlatabiliyorsan kodu gerçekten anlamaya başlamışsındır.

Kaynak izi [K04, K11, K15–K17, K24] Katalog, art, build ve normalizasyon ilişkileri.

Benzer bir uygulamayı sıfırdan yazmak

Bütün LumaNote'u tek seferde yeniden kurmaya çalışmak yerine küçük çalışan aşamalar kullan. Aşağıdaki sıra bir eğitim önerisidir; son kaynak paketin geliştirme geçmişinin birebir kaydı değildir.

Aşama	Üretilcek küçük sonuç	Geçiş ölçütü
1. Statik ekran	HTML/CSS ile başlık, not listesi ve + düğmesi.	Dar ekranda okunuyor, taşmıyor.
2. Veri ve olay	Not ekle, seç, değiştir, sil.	Kimlikler doğru, boş veri reddediliyor.
3. Yerel kayıt	Kayıt ve tekrar açma.	Hata mesajı var, veri kaybı sessiz değil.
4. Zaman mantığı	Başlat, duraklat, bitir, tek sonuç.	Geçen süre ve fazlar tutarlı.
5. Ekonomi	Oturum kredisi ve satın alma.	Çift ödül ve yetersiz bakiye reddediliyor.
6. Piksel sahne	Bir bitki, bir hayvan, sınırlı kamera.	Ekran/dünya koordinatları ayrılmış.
7. Telefon bağlantısı	WebView köprüsü ve native dosya servisi.	Mesaj id, hata ve yaşam döngüsü sınanmış.
8. Genişleme	Odalar, mevsimler, ses ve PDF.	Her ekleme eski kayıtlarla test edilmiş.

Her aşamada dört küçük belge

Ekranın amacı, veri modeli, olay listesi ve beklenen hata davranışını yaz. Örneğin "Satın al" için: girdiler ürün kimliği ve bakiye; başarılı sonuç tek alışveriş kaydı; başarısız sonuç değişmemiş cüzdan. Bu açıklama daha sonra testin ve kodun iskeleti olur.

İlerlemeni nasıl ölçersin?

Dosya veya satır sayısı ile değil, açıklayabildiğin davranışlarla ölç. "Bir notu neden id ile buluyorum?" ve "sayaç neden Date.now kullanıyor?" sorularını cevaplamak, büyük bir kod parçasını ezberlemekten daha yararlıdır.

Kaynak izi Eğitim amaçlı önerilen öğrenme sırası; kaynak mimarisiyle bağlantı: [K02–K24].

“Projem bitti” ve “yayına hazır” ayrımı

İstediğin ürün özelliklerinin tamamlanması, geliştirme sürecinde anlamlı bir dönüm noktasıdır. Bu rehber ürününü tanıtır ve mevcut kodunu açıklayarak devralınabilir hâle getirir. Buna karşılık herkesin indireceği dağıtım, cihaz testi ve yayın yapılandırması ayrı doğrulama işleri içerir.

Başlık	Bu kaynak ne gösteriyor?	Yayın öncesi ayrı doğrulama
Geliştirme bütçesi	100 milyon test cüzdanı için ayrı kapı ve kapatma akışı.	Üretim açılışında test hakkı ve test alışverişleri.
Kayıt	Yerel state.json ve ek dosyalar.	Gerçek cihaz yeniden başlatma, yedek/geri yükleme, düşük depolama.
Bildirim / yön	Expo modül çağrıları var.	İzin reddi, kilit ekranı, yön değişimi.
Not / PDF	Editör ve çizim motorları var.	Gerçek klavye/seçim, büyük dosya, dışa aktarma.
Ses	Paketlenmiş sentezler ve kaynak kayıtları.	Dinleme, ses geçişi, lisans dosyası tutarlılığı.
Dağıtım	Expo kaynak ve yerel başlatma akışı.	Mağaza kimlikleri, imzalama, uygun dağıtım yapısı.

Bu paketi açıklarken bulut hesabı, otomatik cihazlar arası eşitleme veya arka planda kesintisiz ses gibi bulunmayan özellikleri ürün vaadine ekleme. Test raporundaki “yapılmayanlar” bölümünü de geliştirme tesliminin parçası olarak koru.

Güvenlik hakkında ölçülü olmak

HTML temizleyici, mesaj doğrulama, dosya sınırı ve şifreli yedek koruma katmanlarıdır. Bunları “hiç açık yok” garantisi diye sunma. Kullanıcıya gerçek kişisel veri eklenmeden önce izinler, veri açıklaması ve hata yolları ayrıca gözden geçirilmelidir.

Kaynak izi [K01, K02, K05, K06, K25–K29] Kaynak sınırlarından çıkarılan devir kontrol listesi; yeni yayın onayı değildir.

Sık karşılaşılabileceğin sözcükler • 1

Sözcük	Sade açıklama	LumaNote örneği
Arayüz	Kullanıcının gördüğü ve dokunduğu bölüm.	Not listesi ve bahçe ekranı.
Durum / state	O anda uygulamanın bildiği değerler.	S.notes, S.timer.
Model	Verinin hangi alanlardan oluştuğu.	Notun id, title, html alanları.
Bileşen	Arayüz veya davranışın tekrar kullanılan parçası.	Native AudioEngine.
Render	Veriden görünüm üretme/çizme.	renderGarden veya Canvas çizimi.
DOM	HTML öğelerinin tarayıcıdaki nesne ağacı.	Düğmeyi querySelector ile bulmak.
Event / olay	Dokunma, yazma, bırakma gibi bildirim.	pointerdown, input.
Selector	Belirli öğeleri seçmek için ifade.	.note-card gibi CSS seçicisi.
Bridge / köprü	İki çalışma alanının mesaj yolu.	Native.call → onMessage.
Promise	Daha sonra bitecek işin sonucu.	Dosya kaydının tamamlanması.
Debounce	Peş peşe çağrılar kısa süre bekleyip toplama.	Yazarken 260 ms kayıt erteleme.
Normalize	Eksik/geçersiz/eski veriyi beklenen düzene getirme.	Eski odayı salona eşlemek.
Migration / geçiş	Veriyi yeni sürümün modeline uyarlama.	Yeni oda atamaları.
ID / kimlik	Aynı kaydı adı değişse de tanıyan değer.	Not ve oturum kimliği.

Bu sözlükteki benzetmeler öğrenmek içindir. Bir terimin daha ileri teknik ayrıntısı olabilir; önce projedeki rolünü anlamak yeterlidir. Aynı kelimenin veri modeli, çizim ve telefon katmanında hangi anlama geldiğini bağlamıyla oku.

Kaynak izi Kaynak kullanımı: [K02–K24]. Genel açıklamalar bu rehberin öğretici anlatımıdır.

Sık karşılaşılabileceğiniz sözcükler • 2

Sözcük	Sade açıklama	LumaNote örneği
Canvas	Kodla çizim yapılan yüzey.	Bahçe, el yazısı ve PDF sayfası.
Sprite	Küçük oyun nesnesinin çizim/görsel tarifi.	Tavşan veya saksı modeli.
Hit test	Dokunulan noktada hangi nesnenin olduğunu bulma.	Seranın kapısına dokunma.
Pan / zoom	Görünümü kaydırma / büyütme.	Yakınlaşınca bahçede gezinme.
Shader	Görüntü rengi hesaplamak için GPU programı.	Lava efekti.
Fallback	İlk yol kullanılamazsa yedek yol.	Canvas lava çizimi.
Lifecycle	Kurulma, çalışma, kapanma ve temizleme süreci.	AppState dinleyicisini kaldırma.
Manifest	Dosya, sürüm veya varlıkların açıklayıcı listesi.	Audio-Included.json.
Hash	İçeriğin parmak izi gibi hesaplanan değer.	Güncelleyici dosya kontrolü.
Base64	İkili veriyi metinle temsil etme.	Yedekte ek dosyalar. Şifreleme değildir.
Salt / nonce	Şifreleme akışındaki farklı amaçlı rastgele değerler.	Yedek dosyasının kripto parametreleri.
Crossfade	Biri azalırken diğerini artıran ses geçişi.	MP3 döngüsünü yumuşatma.
Regression	Yeni değişiklikten sonra eski işin bozulması.	Oda eklerken not onayının bozulması.
Idempotent	Tekrar uygulamanın ek yan sonuç üretmemesi.	Aynı oturuma yeniden jeton vermemek.

Ezberden çok bağlantı

Bir kelimeyi duyduğunda “hangi dosyada örneğini gördüm?” diye sor. Crossfade için AudioEngine, normalize için model dosyaları, hash için repair-existing aklında canlanıyorsa terim işlev kazanmıştır.

Kaynak izi Kaynak kullanımı: [K04–K29]. Bu kısa sözlük kapsamlı kriptografi veya grafik programlama eğitimi değildir.

Projeyi gerçekten anladın mı?

Soru	Beklenen açıklama
Bahçe sayfası bir React Native View dizisi mi?	Hayır. Native kabuk WebView taşır; bahçe HTML/JS/Canvas tarafında çizilir.
UI_HTML.ts dosyasını düzenlemek neden geçici?	build:ui bu dosyayı tools/ui kaynaklarından yeniden üretir.
Expo hesabı notları buluta yedekler mi?	Bu kaynakta öyle bir LumaNote hesap/senkron servisi yoktur.
Duraklatınca jeton verilmeli mi?	Hayır. Kaydedilmiş etkin çalışma üzerinden tek ödül oluşturulur.
Saksiya ekimden önceki süre ne olur?	Ekim anı toplamı ayrılır; yeni büyüme sonraki çalışma artışından hesaplanır.
Sakura modu bütün Japon dekorlarını verir mi?	Hayır. Atmosfer, katalog sahipliği ve yerleştirme ayrı işlemlerdir.
Arşiv ile çöp kutusu aynı mı?	Hayır; farklı alan ve görünüm kurallarıdır.
Canvas nesnesine dokunma otomatik mi?	Hayır. Ekran noktası dönüştürülüp hit test yapılır.
Şifreli yedek canlı veriyi de şifreler mi?	Bu kaynakta hayır; dışa aktarılan dosya şifrelenir.
MP3 testten geçtiyse sesi doğal mıdır?	Sayısal bütünlük, öznel dinleme kalitesi değildir.
Odalar arasında taşıma nasıl çoğaltılmaz?	Tek ögenin oda ataması/konumu değiştirilir; yeni sahiplik oluşturulmaz.
190 kontrol bütün telefonları kapsar mı?	Hayır. Raporun ortamı ve yapılmayanları ayrıca okunur.

Cevapları görmeden önce kapatıp kendi cümlele anlatmayı dene. Ardından kaynakta ilgili sembolü bul. Açıklaman doğruysa ama dosyayı bulamıyorsan dosya haritasına; dosyayı buluyorsan ama mantığını açıklayamıyorsan kavram bölümüne geri dön.

Kaynak izi Yanıtların kaynakları önceki bölümlerde [K01–K29] ile gösterildi.

Kodda karşılığını bul • 1

K numaraları, sayfaların altındaki proje kaynaklarını eşleştirir. tools/ui ile başlayan bir satırın devamındaki kısa dosya adları aynı klasördedir; nocturne satırlarında da aynı kısaltma kullanılır. Kaynak kopyasında metin aramasıyla işlev adını bulabilirsin.

K01 Proje kimliği ve yapılandırma

package.json; app.json; index.js; babel.config.js; metro.config.js

Sürüm, paketler, komutlar ve Expo giriş noktası.

K02 Native kabuk

App.tsx; nocturne/AudioEngine.tsx

Application, onMessage, send; React yaşam döngüsü ve WebView.

K03 Temel web uygulaması

tools/ui/core.js; events.js; index.html; style.css

S/UI; Native.call; persist; normalize; render/route; ortak olaylar.

K04 Arayüz üretimi

tools/build-ui.cjs

CSS/JS sırası; main/native çıktısı; ses ve PDF paketleme.

K05 Telefon servisleri

nocturne/NativeServices.ts

load/save; dosya seçme; paylaşma; hatırlatma; backup aşamaları.

K06 Tipler ve şifreli yedek

nocturne/types.ts; BackupCrypto.ts; NobleCrypto.ts

AppData; isAppData; encryptBackup/decryptBackup.

K07 Temel ekran ve editör

tools/ui/screens.js; editor.js; editor-v04.js

renderHome; editör; şablonlar; temel sayfa içerikleri.

K08 Güncel not etkileşimleri

tools/ui/living-notes.js; polish-notes.js; polish-shell.js

Sade not listesi; seçim; zengin metin araçları; toplu işlemler.

K09 Çizim ve PDF

tools/ui/living-notes.js; living-pdf.js

Kalem noktaları; PDF sayfa çizimi; işaretleme ve görüntü tabanlı export.

K10 Çalışma motoru

tools/ui/focus.js; living-focus.js; stable-focus.js

remainingMs; startFocus; togglePause; endFocus; mola düzeni.

Kaynak izi Kaynak: LumaNote_v057_Evim_ve_Kis_Bahcem_Tam_Proje.zip. Bu rehber kaynak dosyaları değiştirmez.

Kodda karşılığını bul • 2

K numaraları, sayfaların altındaki proje kaynaklarını eşleştirir. tools/ui ile başlayan bir satırın devamındaki kısa dosya adları aynı klasördedir; nocturne satırlarında da aynı kısaltma kullanılır. Kaynak kopyasında metin aramasıyla işlev adını bulabilirsin.

K11 Dünya ve ilerleme

tools/ui/world.js; world-ui.js

PETS/SEEDS/SCENERIES; syncWorld; plantProgress; sahiplik.

K12 Bahçe temeli

tools/ui/living-garden.js; stable-garden.js; sprites.js

LivingGarden; etkileşim, görünülük ve ilk piksel modeller.

K13 Genişletilmiş yaşam modeli

tools/ui/living-model.js

newLiving; wallet yeniden tanımı; büyüme ve ek kayıtlar.

K14 Temalar ve efektler

tools/ui/lava.js; scenes.js; atelier-themes.js

Lava shader/fallback; efekt çizicileri; tema paletleri.

K15 Mevsimler ve Sakura kataloğu

tools/ui/polish-model.js; polish-art.js; sakura-collection.js; sakura-art.js; sakura-garden.js

Atmosferler; dekor kayıtları; piksel modeller ve özel etkileşimler.

K16 Atölye katmanı

tools/ui/atelier-model.js; atelier-art.js; atelier-garden.js

Çay/Kilim; memorySection056; anı konumu; sınırlı kamera.

K17 Ev, sera ve son katalog

tools/ui/haven-model.js; haven-art.js; haven-garden.js

SPACES057; rooms057; itemSpace057; HomeGarden057; yeni modeller.

K18 Ortak menü ve arayüz katmanları

tools/ui/living-shell.js; stable-shell.js; polish-shell.js; haven-shell.js

sheet/closeSheet; sürükleyerek kapatma; inert ve seçim çubuğu.

K19 Takvim

tools/ui/planner.js; nocturne/NativeServices.ts

planSheet; tekrar örnekleri; takvim dışı aktarma ve bildirimler.

K20 Haftalık özet

tools/ui/weekly.js

weeklyStats; mondayFor; renderWeekly; weeklyToNote.

Kaynak izi Kaynak: LumaNote_v057_Evim_ve_Kis_Bahcem_Tam_Proje.zip. Bu rehber kaynak dosyaları değiştirmez.

Kodda karşılığını bul • 3

K numaraları, sayfaların altındaki proje kaynaklarını eşleştirir. tools/ui ile başlayan bir satırın devamındaki kısa dosya adları aynı klasördedir; nocturne satırlarında da aynı kısaltma kullanılır. Kaynak kopyasında metin aramasıyla işlev adını bulabilirsin.

K21 Yedekleme arayüzü

tools/ui/backup.js

renderBackup; içerik sayıları; önizleme ve geri yükleme onayı.

K22 Ses arayüzü ve oynatma

nocturne/AudioEngine.tsx; AudioAssets.ts; tools/ui/living-audio.js; polish-audio.js; atelier-audio.js; haven-audio.js

Kanallar; kaydırıcılar; piyano seçimi; çapraz geçiş.

K23 Tarayıcı hizmet karşılıkları

tools/ui/browser.js

Native köprü yokken webAction; tarayıcı dosya/ses/kayıt yolları.

K24 Kurulum ve korumalı güncelleme

BASLAT.command; MEVCUT_PROJEYI_DUZELT.command; tools/setup.cjs; repair-existing.cjs; check-port.cjs

Node çağırısı; proje kontrolü; kaynak hashleri; ZIP yedeği; geri alma.

K25 Test kaynakları ve doğrulayıcı

tools/verify.cjs; tests/057; docs/TEST_RAPORU_057.md

Test senaryoları, sonuçları ve açık kapsam sınırlamaları.

K26 Teslim kapsamı

README_TR.md; docs/SOURCE.json; kaynak ZIP

İncelenen sürüm ve paketin kendi açıklamaları; bu rehberin yeni kontrolleri ayrıca etiketlendi.

K27 Ses kaynak kayıtları

licenses/Audio-Included.json; assets/atelier içindeki manifest ve nota dosyaları

Sentez/kayıt ayrımı; ses kimlikleri ve dosya bütünlüğü.

K28 Üretim araçları

tools/generate-assets.py; generate-atelier-audio.py; generate-haven-art.py; generate-piano-057.py

Ses üretimi ve model aracı; telefonda çalışan sayfa kodu değildir.

K29 Geliştirme jetonu

tools/ui/test-coins.js; tools/toggle-test-coins.cjs; TEST_JETONLARI.command

Test bütçesi, native izin, satın alma izi ve kapatma yolu.

Kaynak izi Kaynak: LumaNote_v057_Evim_ve_Kis_Bahcem_Tam_Proje.zip. Bu rehber kaynak dosyaları değiştirmez.

Kavramlar için başvuru kaynakları

Ürüne ilişkin açıklamaların ana kaynağı LumaNote kodudur. Aşağıdaki resmî belgeler yalnızca dil ve araç kavramlarını anlamak için kullanıldı. Bu belgelerde sonradan farklı sürüm örnekleri bulunabilir; proje package.json hedeflerini otomatik değiştirmek için gerekçe sayılmamalıdır.

D01 Expo • Overview of the development process

Expo Go, native katman ve geliştirme süreci.

D02 React Native WebView • Guide

postMessage, onMessage ve web/native iletişim.

D03 TypeScript • TypeScript in 5 minutes

JavaScript'e eklenen tür kontrolünün başlangıcı.

D04 React Native • Core Components and Native Components

View, Text ve native bileşen kavramı.

D05 MDN • Canvas API

Kodla iki boyutlu çizim yüzeyi.

D06 npm • package-lock.json

Kilit dosyasının paket ağacını kaydetme amacı.

D07 Metro • Concepts

Çözümleme, dönüştürme ve paketleme kavramları.

Görseller ve öğretim örnekleri

Sayfa görüntüleri kaynak HTML'den bu rehber için örnek verilerle oluşturuldu. Numaralı işaretler ve mimari şemalar rehber açıklamasıdır. Ö01–Ö04 örnekleri bağımsız eğitim dosyalarıdır. Referans fotoğrafları, filigranlı stok görseller veya font dosyaları bu teslimin varlık paketi olarak paylaşılmıyor.

Okuma tarihi: 1 Ekim 2026. Bağlantılar Word/PDF içinde tıklanabilir. Buradaki kaynakça, gerçek iPhone testini veya ses lisanslarının her yeni dosya için ayrı incelenmesini ikame etmez.

Kaynak izi Resmî ek kaynaklar ürün kodu ile karıştırılmadan sunulmuştur.

Artık projene hangi gözle bakabilirsin?

Başta gördüğün şey bir not defteri, bir sayaç ve küçük bir bahçeydi. Şimdi her birinin kayıt, olay, görünüm ve telefon hizmeti katmanlarından oluştuğunu biliyorsun. Bu ayırım, bir hata çıktığında hangi dosyaya bakacağını ve yeni özellik için neleri birlikte değiştirmen gerektiğini gösterir.

Kendi hızında 12 öğrenme oturumu

Oturum	Odaklanacağın sonuç
1-2	Ürün turunu incele; beş ana ekranın birbiriyle ilişkisini çiz.
3-4	HTML/CSS ve JavaScript örneklerini değiştir; ilk not atölyesini çalıştır.
5-6	Nesne/dizi/id, async/await ve TypeScript türlerini kendi cümlele anlat.
7-8	Başlatma, build, mesaj köprüsü ve kayıt dosyası yolunu izle.
9-10	Sayaç ve mini bahçe atölyelerinde birer kabul testi yaz.
11	Kaynakta bir ürünün katalog → model → satın alma → konum yolunu bul.
12	Küçük bir değişiklik planı ve geri alma/test listesi hazırla.

Bu oturumlar sınav veya zorunlu süre değildir. Bir kavramı anlamadığında önce küçük örneğe dön; sonra gerçek dosyada aynı yapıyı ara. En iyi ilerleme göstergesi, “bu kod ne yapıyor?” sorusuna yalnızca satırı okumak yerine veri akışını açıklayabilmendir.

Projeyi devralan kişiye kısa not

Önce package.json, App.tsx, build-ui.cjs ve core.js'i oku. Ardından değiştirilecek özelliğin en son katmanını bul. Native ve tarayıcı testlerini ayır. Kişisel veriyi, test bütçesini ve kaynak hashlerini bilinçli koru. Bu belgeye göre değil, son etkin kod ve gerçek sonuçlara göre karar ver.

LumaNote'un kısa teknik özeti

React Native/Expo kabuğu + WebView içinde HTML/CSS/JavaScript arayüz + mesaj köprüsü
TypeScript hizmetleri + yerel JSON/dosya saklama + Canvas/SVG piksel dünya + paketlenmiş sesler.
Projeyi anlamanın omurgası budur.

Kaynak izi Belge, 0.5.7 kaynak snapshot'ının eğitim amaçlı açıklamasıdır; uygulama kodunda değişiklik yapılmadı.